Introduction

This book documents the **Serai network** — the decentralized execution layer — treating the **Serai DEX** as what the protocol's own specification calls it: "the premier application of Serai," not the network itself.

Why this book exists

Serai describes itself two incompatible ways.

> "Serai is a new DEX, built from the ground up … offering a liquidity-pool-based trading experience." — `code/README.md:3-6`

> "Serai is a decentralized execution layer whose validators form multisig wallets for various connected networks, offering secure decentralized control of foreign coins to **applications built on it**. Serai is exemplified by Serai DEX … **the premier application of Serai**." — `code/spec/Serai.md:1-9`

The second framing — Serai as a general execution layer with the DEX as one application — is **asserted but never specified**. There is no application model, no second application, and no plugin or SDK surface anywhere in the tree; the one page in the project's own docs where an integration model would live, `code/docs/integrating/index.md`, is an empty front-matter-only stub. The published documentation (`code/docs/`) is a partially-stubbed Jekyll site whose pages on cross-chain architecture, per-era economics, validator operation, and integration are blank or placeholder.

So the gap this book fills is precise: **nowhere does the project explain where "Serai the network" ends and "the DEX application" begins.** Answering that

— from the code, honestly — is the spine of this book, and its dedicated synthesis is Part V: Where Serai Ends and the DEX Begins.

What we found, in one paragraph

Serai *is* architected as the spec claims: a generalized primitive of **cross-chain custody plus programmable "in-instructions"** — a signed cross-chain deposit can carry an instruction the runtime executes atomically on arrival — and the plumbing that delivers it (the off-chain `Network` abstraction, the coordinator, the `execute_batch` pipe, the `Coins` ledger) never mentions the DEX. But as *shipped*, the boundary is only nominal. The instruction set is a closed on-chain enum in which only `Transfer` is application-free; three of its four variants invoke the DEX. And the network's own economic-security math — the rule deciding how much foreign value its multisigs may hold — is load-bearing on the DEX's AMM price oracle. The two are separable in **concept and code layering**, yet **economically inseparable at runtime**. This book documents both the clean generalization and the places the DEX is wired back into the network's core.

How to read it

The book is **layered**:

- **Part I — Concepts** is prose-first and assumes no prior knowledge: what Serai is, how the whole system fits together, and a glossary.
- **Parts II–V** are a **reference layer**. Nearly every non-obvious claim carries a `code/<path>:<line>` citation into the pinned source, so the book doubles as an audit companion and can be checked against the code line by line.

The boundary legend

Every subsystem in the reference layer is tagged so the network/application split is legible on every page:

Tag	Meaning
`[NETWORK-CORE]`	Part of the generalized Serai network. Independent of any application.
`[DEX-APP]`	Belongs to the DEX reference application. Documented only at its interface — no AMM internals.
`[COUPLED]`	Nominally network, but at runtime depends on the DEX (e.g. the price oracle). Called out explicitly.

Source of truth

The source of truth is the target repository at commit `4b89cf0206184886e96d0663861596312e5b47d2` — the git submodule under `../code/`, pinned in `../work/manifest.json`. All `code/…` paths and line numbers are relative to that commit. Where the repository's own prose (`code/docs/`, `code/spec/`) disagrees with the code, **the code is treated as authoritative** and the discrepancy is flagged inline and collected in the Networks & Coins Reference appendix.

> **Scope.** This book documents the network. It describes the DEX only where it forms the network's application boundary or couples into the network's security and monetary policy. It does **not** document the AMM's internal mechanics (pool math, LP accounting, swap routing) — those belong to a Serai DEX manual, not a Serai network manual.

What Serai Is

Two answers to one question

Ask “what is Serai?” and the repository gives two different answers.

The `README` answers **application-first**:

> “Serai is a new DEX, built from the ground up, initially planning on listing Bitcoin, Ethereum, DAI, and Monero, offering a liquidity-pool-based trading experience. Funds are stored in an economically secured threshold-multisig wallet.” — `code/README.md:3-6`

The protocol specification answers **network-first**:

> “Serai is a decentralized execution layer whose validators form multisig wallets for various connected networks, offering secure decentralized control of foreign coins to **applications built on it**. Serai is exemplified by Serai DEX … **the premier application of Serai**.” — `code/spec/Serai.md:1-9`

Both are true, and the tension between them is the reason this book exists. This book takes the **network-first** view: its subject is Serai the execution layer, and the DEX is treated as what the spec calls it — the premier *application*, not the platform. Where the DEX is unavoidable (because the shipped network leans on it), we document that honestly rather than pretend a clean separation that the code does not have. The rigorous, cited version of that boundary is Where Serai Ends and the DEX Begins; this chapter is the plain-language orientation.

The generalized primitive

Underneath the DEX, Serai offers one primitive: **decentralized cross-chain custody with programmable in-instructions.**

- A set of validators stakes the native coin, **SRI**, and collectively forms **threshold multisig wallets** on external chains. No single validator controls the keys; signing requires a threshold of them cooperating (see Cryptography and Validator Sets).
- When someone **deposits** to one of these wallets, the deposit can carry an opaque data payload. Off-chain infrastructure decodes that payload into an **in-instruction** — a small, typed command — and the Serai blockchain executes it **atomically when the deposit is confirmed**. The simplest in-instruction just mints a 1:1 on-chain representation of the deposit to a Serai account; richer ones hand the funds to an application.
- To exit, a user **burns** their on-chain representation, which instructs the validators to produce a threshold-signed **withdrawal** on the external chain.

That deposit → instruction → execute → burn → withdraw loop is the whole network in miniature, and it is genuinely application-agnostic in its plumbing. The off-chain code that watches chains and signs transactions never mentions the DEX; it just moves value and delivers opaque instructions.

Real, but narrow

Is Serai actually a *general* platform, then? The honest answer is: the generalization is real but narrow, as it ships.

- The instruction the network can execute is a **closed enum**, not an open registry. Exactly one of its variants (`Transfer` , a plain bridge-in) is application-free; the rest invoke the DEX.
- The network's own **economic-security** rule — how much foreign value a multisig may safely hold — is computed from the DEX's price oracle. The security model literally depends on the application.

So Serai is separable from the DEX in concept and in code layering, but as deployed the two are economically inseparable. The boundary chapter walks every coupling with citations; keep the caveat in mind as you read Parts II–IV.

What it deliberately is *not*

Serai is not a token-listing venue. The code caps coins per network at 8 and comments that this is a hedge, not an ambition:

> "Since Serai isn't interested in listing tokens, as on-chain DEXs will almost certainly have more liquidity, the only reason we'd have so many coins from a network is if there's no DEX on-chain." — `code/substrate/primitives/src/networks.rs:395-401`

The initial connected assets are **Bitcoin (BTC), Ether (ETH), DAI, and Monero (XMR)** (`code/docs/index.md:15-16`) — a small, curated set, each mapped to a fixed external network. Adding another chain or coin is a protocol fork, not a listing (see External Network Integrations).

It is also **fairly launched**. The native coin SRI does not exist before mainnet, and there is no ICO, IEO, presale, developers' fund, or airdrop for off-mainnet activity — "all distributions of SRI will happen on-chain per the protocols' defined rules, based on on-chain activity" (`code/docs/index.md:21-32`). The economics of that distribution are covered in SRI Tokenomics & Emissions.

How this book is organized

- **Part I — Concepts** (you are here): the System Architecture at a glance, and a Glossary of the vocabulary the rest of the book assumes.
- **Part II — The Serai Blockchain**: the on-chain runtime — the ledger, validators and staking, economic security, the instruction interface, governance, and tokenomics.

- **Part III — Off-Chain Infrastructure**: the processor, coordinator, message bus, and external-chain integrations.
- **Part IV — Cryptography**: the threshold-signing stack, key generation, and FROST signing.
- **Part V — The boundary**: Where Serai Ends and the DEX Begins.

A note on trust as you read: the source of truth is the code at the pinned commit, and nearly every claim in Parts II–V carries a `code/<path>:<line>` citation you can check.

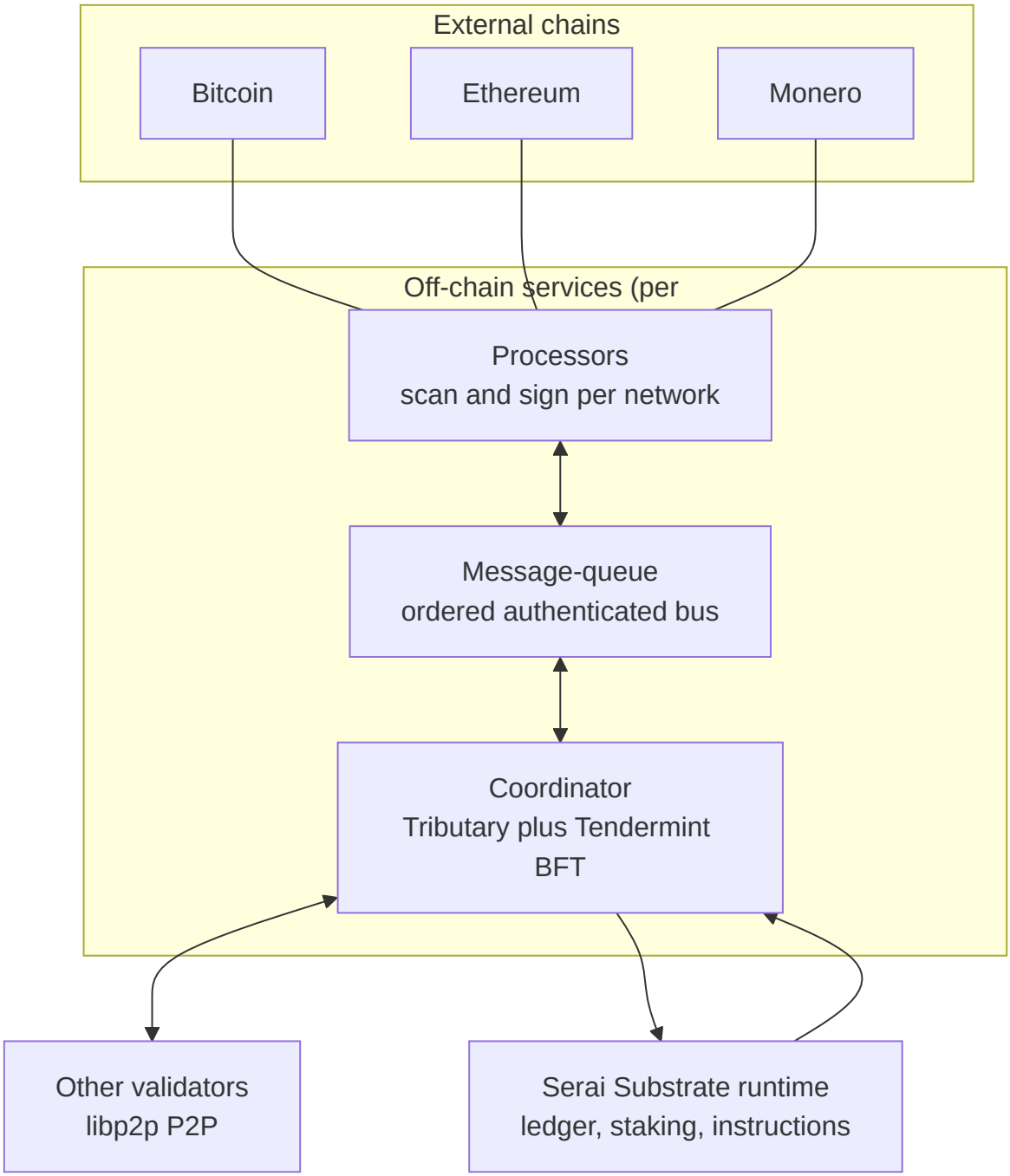
System Architecture

Serai is not one program. It is a small blockchain plus a set of off-chain services that every validator runs, arranged in layers between the external chains it custodies and the Serai chain that accounts for that custody. This chapter shows how the pieces fit and follows a unit of value all the way through; the deep treatment of each piece is in Parts II–IV.

The layers

From the outside in:

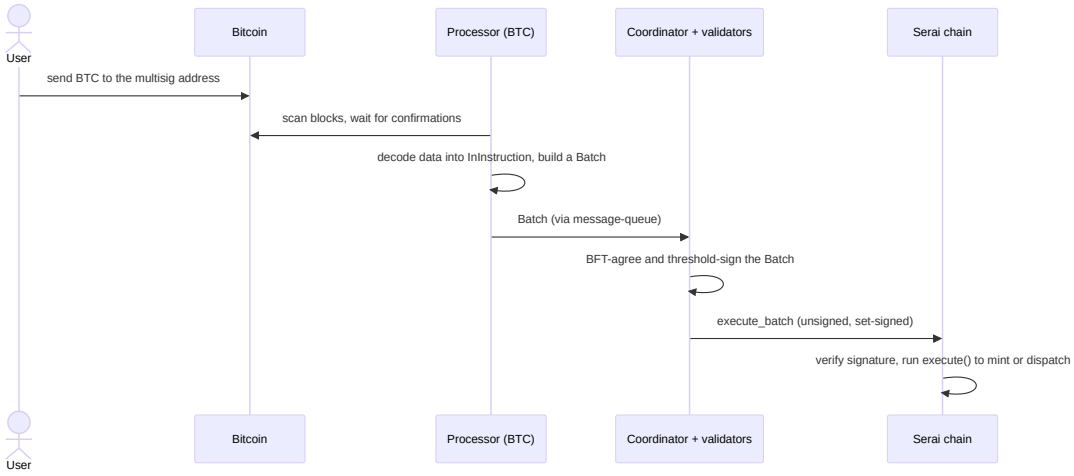
- **External chains** — Bitcoin, Ethereum, Monero. Where user funds actually live, in Serai-controlled threshold multisigs.
- **Processors** — one per external network. A processor scans its chain for deposits, turns them into `Batch` es of instructions, and — when told to — builds and threshold-signs outbound transactions. It holds the network's key shares and never talks to the Serai chain directly. `[NETWORK-CORE]`
- **Message-queue** — an ordered, authenticated message bus. Every message flows between a processor and the coordinator; the queue guarantees ordering and survives peers going offline (`code/README.md:31-32`). `[NETWORK-CORE]`
- **Coordinator** — one per validator. The coordinator is the brain: it reads finalized Serai blocks, drives distributed key generation and signing, and reaches Byzantine-fault-tolerant agreement with the other validators over a peer-to-peer network. It does this by running, for each active validator set, a dedicated ephemeral blockchain called a **Tributary**, itself driven by a from-scratch **Tendermint** consensus engine. `[NETWORK-CORE]`
- **Serai Substrate runtime** — the Serai blockchain: the ledger, staking, economic security, the instruction interface, governance, and emissions. This is the source of truth for who staked what and which coins are custodied. `[NETWORK-CORE]`



The DEX does not appear in this diagram. Everything shown is network infrastructure; the DEX is a set of pallets *inside* the Serai runtime, reached only after an instruction has been executed. That is the architectural sense in which Serai is “a network with the DEX on top” — see the boundary chapter.

Following value in: a deposit

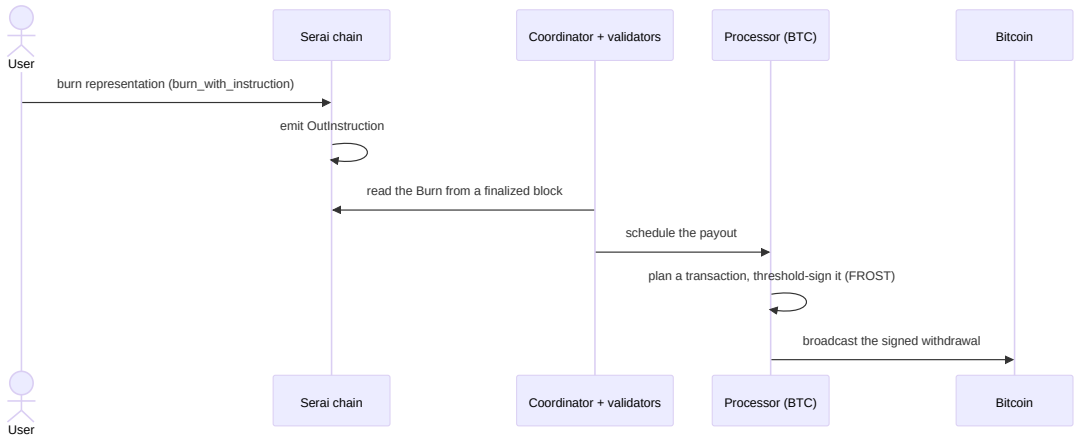
Suppose someone deposits Bitcoin to be represented on Serai.



The processor only treats a deposit as real after a chain-specific number of confirmations, and it decodes the deposit's opaque bytes into a typed `InInstruction` **without interpreting application meaning** — the meaning is assigned later, on-chain. The validators agree on the batch through their Tributary and collectively sign it; the Serai runtime accepts the batch as an *unsigned* extrinsic whose authority is the validator set's threshold signature, then runs the instruction (see The Cross-Chain Instruction Interface). The most basic instruction mints a 1:1 representation to a Serai address; others hand the funds to the DEX.

Following value out: a withdrawal

Exiting reverses the flow.



Burning an on-chain representation produces an `OutInstruction`; the coordinator observes it in a finalized Serai block and tasks the relevant processor, which plans a concrete transaction and drives a FROST threshold signing across the validators before broadcasting the result. No individual validator can move custodied funds; only a threshold acting together can.

A cross-cutting liveness note

Every one of these services installs a panic hook that calls `process::exit(1)` on any unhandled panic. That makes the code fail loudly and safely, but it also means any attacker-reachable `unwrap!` / `panic!` on network, RPC, or peer input is a potential remote liveness (denial-of-service) concern. The pattern recurs across the processor, coordinator, and message-queue, and is called out where relevant in Part III. It is a design theme worth carrying as you read.

Where to go next

- The on-chain half: start at Runtime & Call Surface.
- The off-chain half: start at Overview & the Message Bus.
- The vocabulary used everywhere: the Glossary.

Glossary

Vocabulary the rest of the book assumes. Each entry points at the code where the concept is defined or enforced. Classifications match the boundary chapter.

AllowMint

The gate deciding whether a bridged (external) coin may be minted. The runtime wires it to `ValidatorSets`, so a mint is refused unless staked value covers the DEX-priced economic-security requirement (`code/substrate/validator-sets/pallet/src/lib.rs:1185-1196`; wiring at `code/substrate/runtime/src/lib.rs:204-206`). `[COUPLED]` — see Economic Security.

Allocation / Deallocation

Adding stake to (`allocate`) or scheduling its removal from (`deallocate`) a validator's position in a set. Deallocations have a cooldown and cannot drop a set below its fault-tolerance or economic-security floor (`code/substrate/validator-sets/pallet/src/lib.rs`). `[NETWORK-CORE]`

Batch

A validator-signed bundle of `InInstruction`s from one external network, produced by a processor and executed on-chain via `execute_batch`. Carries a network id, sequential id, and external block reference (`code/substrate/in-instructions/primitives/src/lib.rs:106-126`). `[NETWORK-CORE]`

Coordinator

The per-validator off-chain service that drives DKG and signing, talks to peers over P2P, and reaches BFT agreement by running a Tributary per validator set (`code/coordinator/`). See The Coordinator. `[NETWORK-CORE]`

Cosign

Validators co-signing finalized Serai blocks so external observers (and each other) can detect a chain split; a supermajority cosigning a *different* chain halts the node (`code/coordinator/src/cosign_evaluator.rs`). `[NETWORK-CORE]`

DKG (Distributed Key Generation)

The protocol by which a validator set jointly generates a threshold key with no party ever holding the whole secret. Serai uses PedPoP (`code/crypto/dkg/`). See Distributed Key Generation. `[NETWORK-CORE]`

Economic security

The property that misbehaving is unprofitable: the stake required to forge unintended signatures must exceed the value obtainable by doing so. A network is flagged “secured” once its DEX oracle exists and the mint gate passes (`code/substrate/economic-security/pallet/src/lib.rs:44-50`). `[COUPLED]`

Eventuality

A processor’s record of an expected on-chain outcome (e.g. a specific signed transaction), used to match broadcast transactions back to the plan that created them and to detect completion
(`code/processor/src/networks/mod.rs`). `[NETWORK-CORE]`

External network

One of the fixed connected chains — Bitcoin, Ethereum, Monero — modeled as `ExternalNetworkId`, a closed 3-variant enum with a hand-rolled codec (`code/substrate/primitives/src/networks.rs:21-47`). `[NETWORK-CORE]`

FROST

The two-round threshold Schnorr signing protocol the validators use to sign external-chain transactions and Serai batches (`code/crypto/frost/`). See FROST Threshold Signing. `[NETWORK-CORE]`

Handover

The rotation of custody from a retiring validator set's multisig to the new set's, including the one-time retirement of the prior set the first time the new key signs a batch (`code/substrate/in-instructions/pallet/src/lib.rs`; `code/substrate/validator-sets/pallet/src/lib.rs`). `[NETWORK-CORE]`

InInstruction

The typed command a deposit can carry, executed atomically on arrival. A closed enum: `Transfer` (bridge-in, `[NETWORK-CORE]`), `Dex(..)` and `GenesisLiquidity` (`[DEX-APP]`), and `SwapToStakedSRI` (`[COUPLED]`) (`code/substrate/in-instructions/primitives/src/lib.rs:77-82`). The extension seam — see the boundary chapter.

Key share

One validator's secret portion of a threshold key. A threshold of shares can sign; fewer cannot. Held by the processor, derived from its `ENTROPY` (`code/processor/src/key_gen.rs`). `[NETWORK-CORE]`

Message-queue

The ordered, authenticated bus carrying every coordinator ↔ processor message, persistent across restarts (`code/message-queue/`). See Overview & the Message Bus. `[NETWORK-CORE]`

MuSig

An n-of-n key-aggregation scheme used to confirm a DKG result on-chain (validators jointly sign the new key with their identity keys)
(`code/crypto/dkg/musig/`). `[NETWORK-CORE]`

PedPoP

Pedersen DKG with proof-of-possession — the specific interactive DKG Serai runs, with encrypted secret shares and a blame protocol for faulty participants
(`code/crypto/dkg/pedpop/`). `[NETWORK-CORE]`

Processor

The per-external-network off-chain daemon: scans the chain for deposits, builds batches, plans and threshold-signs withdrawals, and holds the key shares (`code/processor/`). See The Processor. `[NETWORK-CORE]`

Required stake

The SRI value a set must have staked to safely custody a coin, computed from the DEX price oracle (roughly 1.7× the coin's value in code)
(`code/substrate/validator-sets/pallet/src/lib.rs:833-863`). `[COUPLED]`

Retirement / Halt signal

Governance signals: `Retirement` coordinates a runtime upgrade (and deliberately halts the chain at a scheduled block), `Halt` stops processing for one external network. Both need supermajority stake in favor (`code/substrate/signals/pallet/src/lib.rs`). See Governance. `[NETWORK-CORE]`

Serai address

An account identity on Serai — an `sr25519` public key (32 bytes), wrapped as `SeraiAddress` (`code/substrate/primitives/src/`). `[NETWORK-CORE]`

Session

A period of a fixed validator set. Rotation to a new session re-selects validators, enacts key handover, and drives Babe/Grandpa authority changes (`code/substrate/validator-sets/pallet/src/lib.rs`). `[NETWORK-CORE]`

Slash

A penalty removing (some of) a validator's stake for provable misbehavior (e.g. consensus equivocation). External-network slashing is currently a no-op TODO (`code/substrate/validator-sets/pallet/src/lib.rs`). `[NETWORK-CORE]`

Shorthand

A compact on-the-wire encoding of an `InInstruction` carried in a deposit's data. `Raw` is wired up; the `Swap` / `SwapAndAddLiquidity` sugar is still `todo!()` (`code/substrate/in-instructions/primitives/src/shorthand.rs`). `[NETWORK-CORE]` pipe / `[DEX-APP]` sugar.

SRI

Serai's native coin (`Coin::Serai` , symbol `"SRI"` , 8 decimals). Gas, stake, and settlement asset; minted freely by the runtime (emissions, rewards, genesis) unlike bridged coins (`code/substrate/primitives/src/networks.rs:358-392`). `[NETWORK-CORE]`

Tendermint

The from-scratch Byzantine-fault-tolerant consensus engine driving each Tributary; safe and live under less than one-third Byzantine voting weight (`code/coordinator/tributary/tendermint/`). `[NETWORK-CORE]`

Threshold multisig (t-of-n)

A wallet whose key is split among `n` validators such that any `t` of them can sign but fewer cannot. The basis of Serai's custody; formed by DKG, signed by FROST. `[NETWORK-CORE]`

Tributary

A dedicated, ephemeral BFT blockchain run per validator set to order the inputs of DKG and signing before relaying them to processors (`code/coordinator/tributary/`). `[NETWORK-CORE]`

Validator set

The group of validators securing one network for a session, selected by stake, forming that network's multisig (`code/substrate/validator-sets/pallet/src/lib.rs`). See Validator Sets. `[NETWORK-CORE]`

Runtime & Call Surface

`[NETWORK-CORE]`

The Serai blockchain is a single Substrate/FRAME runtime. This chapter documents its pallet composition, the exact set of transactions it will accept, and how a signed transaction is verified. It is the on-chain foundation everything in Part II builds on; the network/application split introduced here is defended in full in the boundary chapter.

Identity

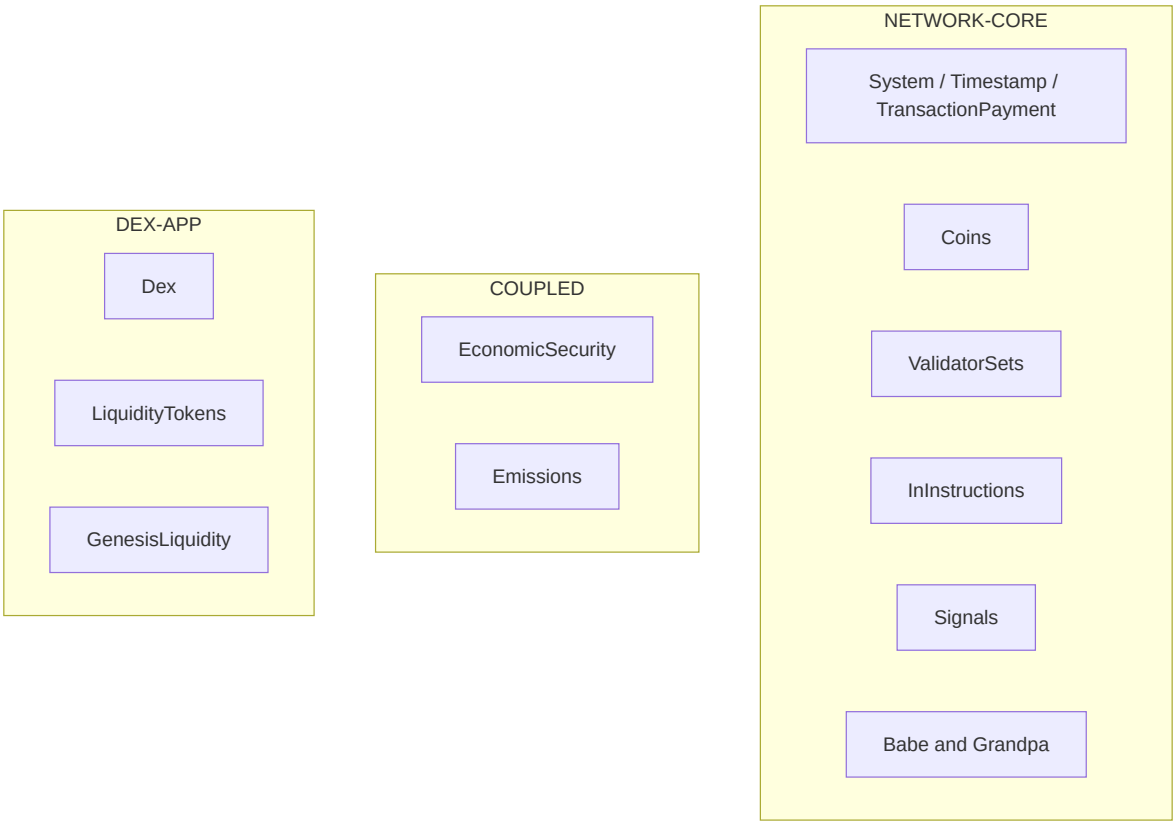
The runtime declares itself `spec_name = "serai"`, `impl_name = "core"` (`code/substrate/runtime/src/lib.rs:109-110`) — the name already encodes the framing this book follows: the chain is *serai*, and the DEX is one thing *core* runs, not the runtime's identity.

Pallet composition

The whole chain is one `construct_runtime!` (`code/substrate/runtime/src/lib.rs:318-343`). There is **no module boundary or application registry** between network pallets and DEX pallets — they sit in one flat list, and the crate name is the only signal of which is which. Classified per the boundary chapter:

Pallet	Class	Responsibility
`System`	NETWORK-CORE	FRAME system
`Timestamp`	NETWORK-CORE	Block time; `OnTimestampSet = Babe` (`:189`)

Pallet	Class	Responsibility
`TransactionPayment`	NETWORK-CORE	Fees; `OnChargeTransaction = Coins` (`:196`)
`Coins`	NETWORK-CORE	Native SRI + bridged balances — see The Coins Ledger
`LiquidityTokens` (`coins::Instance1`)	**DEX-APP**	LP-share token accounting
`Dex`	**DEX-APP**	The AMM and its price oracle
`ValidatorSets`	NETWORK-CORE	Staking, sessions, keys, slashing — see Validator Sets
`GenesisLiquidity`	**DEX-APP**	One-time DEX bootstrap
`Emissions`	**COUPLED**	SRI issuance; post-security split by DEX volume
`EconomicSecurity`	**COUPLED**	Flags a network as economically secured (reads the DEX oracle)
`InInstructions`	NETWORK-CORE	Executes cross-chain deposit batches — see Instructions
`Signals`	NETWORK-CORE	Governance — see Governance
`Babe`	NETWORK-CORE	Block production
`Grandpa`	NETWORK-CORE	Finality



The public dispatch surface: `CallFilter` + `serai_abi::Call`

Serai does not accept arbitrary `RuntimeCall`s. A `CallFilter` is installed as the system `BaseCallFilter` (`code/substrate/runtime/src/lib.rs:151-152`) and admits a call **only if it can be converted into a `serai_abi::Call`** (`:142-149`):

```
fn contains(call: &RuntimeCall) -> bool {
  let call: Result<serai_abi::Call, ()> = call.clone().try_into();
  call.is_ok()
}
```

Therefore the `serai_abi::Call` enum (`code/substrate/abi/src/lib.rs:38-59`) **is** the exact public transaction surface of the chain. Anything not represented there — including `RuntimeCall::System(_)` — is unroutable;

there are no `sudo` /system dispatchables exposed to users. The enum uses explicit `#[codec(index = …)]` tags:

ABI variant (codec index)	Class	Calls
`Timestamp` (1)	NETWORK-CORE	`set`
`Coins` (3)	NETWORK-CORE	`transfer`, `burn`, `burn_with_instruction`
`LiquidityTokens` (4)	**DEX-APP**	`transfer`, `burn`
`Dex` (5)	**DEX-APP**	`add_liquidity`, `remove_liquidity`, `swap_exact_tokens_for_tokens`, `swap_tokens_for_exact_tokens`
`ValidatorSets` (6)	NETWORK-CORE	`set_keys`, `report_slashes`, `allocate`, `deallocate`, `claim_deallocation`
`GenesisLiquidity` (7)	**DEX-APP**	`remove_coin_liquidity`, `oraclize_values`
`InInstructions` (10)	NETWORK-CORE	`execute_batch` (unsigned)
`Signals` (11)	NETWORK-CORE	`register_retirement_signal`, `revoke_retirement_signal`, `favor`, `revoke_favor`, `stand_against`
`Babe` (12)	NETWORK-CORE	`report_equivocation[_unsigned]`
`Grandpa` (13)	NETWORK-CORE	`report_equivocation[_unsigned]`

`Emissions` and `EconomicSecurity` have **no `Call` variant at all** — they are engine-room pallets driven only by block hooks. `TransactionPayment` and `System` are likewise absent from the surface.

Signed-transaction verification

Serai uses a **custom transaction type** (`serai_abi::tx::Transaction<Call, Extra>`), not Substrate's stock `UncheckedExtrinsic` . Verification is a hand-written `BlindCheckable::check` (`code/substrate/abi/src/tx.rs:147-169`):

- A signed transaction carries `(signer, signature, extra)` . `check` verifies the sr25519 signature over `(&self.call, &extra, extra.implicit()?).encode()` (`:152-153`); a bad signature yields `InvalidTransaction::BadProof` .
- The signature commits to `self.call` (the ABI call), but the checked extrinsic dispatches `self.mapped_call` (`:160`). The two are set together in `new()` — `call = mapped.try_into()` , `mapped_call = mapped` (`:113-114`) — so they are two encodings of the same call. That they can never diverge is worth confirming in review, since the signed-over value and the dispatched value differ syntactically.
- Bare (unsigned) transactions skip signature verification and dispatch `mapped_call` directly (`:163-166`); their authorization lives in each pallet's `ValidateUnsigned` (see Instructions and Validator Sets).

Replay and era protection come from `SignedExtra` — `CheckNonce` , `CheckEra` , `CheckGenesis` , `CheckSpecVersion` — folded into the signed payload via `extra.implicit()` .

> **Review note.** During block execution, apply-extrinsic, and transaction validation the runtime auto-sets a signer's `providers` to 1 when it is 0 (`code/substrate/runtime/src/lib.rs:388-389,408-409,441-442`). This account-existence handling is worth checking for any free-account-creation or nonce-reset angle.

Consensus configuration

- **Babe** produces blocks with a 4-week epoch (`EpochDuration = 4 * 7 * DAYS` , `code/substrate/runtime/src/lib.rs:271`), or a short epoch under the `fast-epoch` feature (`:268`) used for testing. Expected block time is `TARGET_BLOCK_TIME` (6 s; see Constants).
- **Grandpa** provides finality.
- **Equivocation reporting** for both Babe and Grandpa is routed into `ValidatorSets` (`:282-283,295-296`), which is also the `DisabledValidators` source (`:275`) and the block author (`pallet_authorship::FindAuthor = ValidatorSets` , `:256`). Slashing therefore lives in the network core, not in the consensus pallets — see Validator Sets.

The `AllowMint` wiring — a first look at the coupling

Two `coins::Config` impls are installed (`code/substrate/runtime/src/lib.rs:204-210`):

```
impl coins::Config for Runtime { type AllowMint = ValidatorSets; }
// main Coins
impl coins::Config<coins::Instance1> for Runtime { type AllowMint =
(); }        // LiquidityTokens
```

For the main `Coins` instance, minting a bridged coin is gated by `ValidatorSets` , whose answer is computed from the **DEX price oracle**. For the `LiquidityTokens` instance, minting is unconditional (`()` always allows). This one line is the seam by which the network's core ledger depends on the application; it is documented in The Coins Ledger and analysed in full in Economic Security.

The Coins Ledger

`[NETWORK-CORE]`

`coins` is the ledger for every asset on Serai: the native token SRI and the bridged representations of external coins. It is where value is minted when a deposit arrives, burned when a withdrawal leaves, and transferred between accounts. Because both the network's bridge and the DEX application move value through it, `coins` is the shared floor both layers stand on — but the pallet itself is network core, and one gate inside it (`mint`) is the single seam where the DEX reaches back into the network.

Coins: native vs bridged

The coin type is a two-variant enum (`code/substrate/primitives/src/networks.rs:254-257`):

```
pub enum Coin { Serai, External(ExternalCoin) }
pub enum ExternalCoin { Bitcoin, Ether, Dai, Monero }   // :193-198
```

- `Coin::Serai` is **SRI** — name "Serai", symbol "SRI", 8 decimals, `is_native() == true` (`networks.rs:369-392`). It is the gas, staking, and settlement asset.
- `Coin::External(_)` are bridge-backed IOUs (`sriBTC` , `sriETH` , `sriDAI` , `sriXMR`), each 1:1 with foreign value held in a Serai multisig.

Coins carry a hand-rolled SCALE codec with explicit tag bytes: `Coin::Serai` encodes to `[0]` (`networks.rs:262`); the external coins encode to `[4]` / `[5]` / `[6]` / `[7]` for Bitcoin/Ether/Dai/Monero (`:200-209`).

> **Spec-vs-code discrepancy.** The prose spec (`code/spec/protocol/Constants.md`) numbers coins Serai=0, Bitcoin=1, Ether=2, DAI=3, Monero=4 (human-facing IDs), whereas the SCALE

`Encode` above uses tags 4–7 for the external coins. Documents and tooling that assume the spec numbering will mis-encode. Collected in Networks & Coins.

Two instances

The pallet is instantiated twice in the runtime (`code/substrate/runtime/src/lib.rs:204-210`):

- **`Coins`** — the main ledger of SRI and bridged coins. `[NETWORK-CORE]`
- **`LiquidityTokens`** (`coins::Instance1`) — LP-share accounting for the DEX. `[DEX-APP]`

They share all the code below; they differ only in their `AllowMint` wiring and in one blocked extrinsic (`burn_with_instruction`).

Storage & the supply invariant

- `Balances: DoubleMap<(Public, Coin) -> u64>` — every account's balance per coin.
- `Supply: Map<Coin -> u64>` — the circulating supply per coin, seeded to 0 for all `COINS` at genesis (`code/substrate/coins/pallet/src/lib.rs:100-104`).

The load-bearing invariant is **`Supply[c] == Σ Balances[*, c]`**. Every mutation maintains it: `mint` increases both a balance and supply (`:177-183`), `burn_internal` decreases both (`:197-201`), `transfer_internal` moves between balances only (`:207-217`). Zero-balance rows are removed (`:142-143`). Amounts are `u64`; internal helpers use `checked_add` / `checked_sub` (`AmountOverflowed` / `NotEnoughCoins`, `:133-161`).

Minting and the `AllowMint` gate

`mint` is the value-creation primitive (`code/substrate/coins/pallet/src/lib.rs:166-187`), and its first act is the gate:

```
if !ExternalCoin::try_from(balance.coin)
  .map(|coin| T::AllowMint::is_allowed(&ExternalBalance { coin,
amount: balance.amount }))
  .unwrap_or(true)                        // Coin::Serai -> try_from
fails -> unwrap_or(true) -> allowed
{ Err(Error::MintNotAllowed)? }           // :167-174
```

- For `Coin::Serai`, `ExternalCoin::try_from` fails, so `unwrap_or(true)` makes minting **always allowed**. SRI is freely minted by the runtime (emissions, rewards, genesis).
- For `Coin::External(_)`, minting must pass `T::AllowMint::is_allowed`. The runtime wires `AllowMint = ValidatorSets` for the main instance (`runtime/src/lib.rs:204-206`), so **a bridged coin can only be minted while staked value covers the DEX-priced required security.** `[COUPLED]`

This is the network↔application seam. Its full mechanics — how `ValidatorSets::required_stake` reads `Dex::security_oracle_value` — are in Economic Security. For the `LiquidityTokens` instance, `AllowMint = ()` allows minting unconditionally (`:208-210`).

Extrinsics

All three are `ensure_signed` (`code/substrate/coins/pallet/src/lib.rs:220-256`):

call_index	Extrinsic	Ef
0	`transfer(to, balance)`	`transfer_inter`
1	`burn(balance)`	`burn_internal`

call_index	Extrinsic	Ef
2	`burn_with_instruction(instruction)`	Bridge-out — se **for** `LiquidityTo` `BurnWithInstru`

Internal cross-pallet primitives (`pub` / `pub(crate)` , no origin check — the caller pallet gates them) are the ones every other pallet uses: `mint` , `burn_internal` , `transfer_internal` , `increase_balance_internal` , `decrease_balance_internal` .

Bridging out

`burn_with_instruction` (`:243-255`) is the withdrawal trigger. The caller burns a bridged balance and supplies an `OutInstructionWithBalance` (`code/substrate/coins/primitives/src/lib.rs:33-36`):

```
pub struct OutInstruction          { pub address: ExternalAddress,
pub data: Option<Data> }   // :24-27
pub struct OutInstructionWithBalance { pub instruction:
OutInstruction, pub balance: ExternalBalance } // :33-36
pub enum Destination { Native(SeraiAddress),
External(OutInstruction) }                        // :42-45
```

The emitted `Event::BurnWithInstruction` is what the off-chain processor observes to build and threshold-sign the actual external-chain payout (see The Processor). Blocking this call for the `LiquidityTokens` instance ensures LP shares cannot be bridged off-chain — they are a purely on-chain DEX artifact.

Fees are SRI, and burned every block

`coins` implements `OnChargeTransaction` (`:258+`): transaction fees are withdrawn in `Coin::Serai` into the well-known `FEE_ACCOUNT` (`system_address(b"Coins-fees")` ,

`code/substrate/coins/primitives/src/lib.rs:18`). Then, at the **start of every block**, `on_initialize` burns the entire fee-account balance (`code/substrate/coins/pallet/src/lib.rs:115-124`):

```
Self::burn_internal(FEE_ACCOUNT.into(), Balance { coin, amount
}).unwrap();
```

SRI is therefore deflationary with respect to fees. Note the `.unwrap()`: the code's own comment (`:120-121`) warns that because it runs at the start of every block, an error here **halts the runtime**. It is expected to be infeasible (you cannot burn more than the account holds), but it is one of several block-hook `unwrap()` / `panic!` sites that make liveness contingent on those invariants — see the enumeration in Governance.

Why this is network core, not application

Nothing in `coins` knows what a swap is. It mints, burns, transfers, and tracks supply for an enum of coins. The DEX, genesis-liquidity, emissions, and the bridge all call the same primitives. The only application-shaped thing about it is the `AllowMint` type parameter — and even that is filled by a *network* pallet (`ValidatorSets`), which merely happens to consult the DEX oracle. `coins` is the ledger of the network; the DEX is one of its users.

Validator Sets, Staking & Sessions

`[NETWORK-CORE]` — `code/substrate/validator-sets/pallet/src/lib.rs`, `code/substrate/validator-sets/primitives/src/lib.rs`.

The `validator-sets` pallet is the heart of the Serai network: it manages who the validators are, how much SRI they have staked, how stake maps to multisig **key shares**, when validator sets rotate, which threshold-multisig **keys** each external network is using, and how misbehaviour is slashed. It is also where the network's custody-safety gate lives — the `AllowMint` implementation documented in Economic Security. Everything here is application-independent; the one line that reaches into the DEX application (`Dex::on_new_session`) is called out below as `[COUPLED]`.

A Serai *validator set* exists per network (`NetworkId::Serai` plus one per external network). The `Serai` set additionally produces and finalizes Serai's own blocks via Babe/Grandpa; the external sets exist to form threshold multisigs over Bitcoin/Ethereum/Monero.

Storage — the staking state

Storage	Shape	Purpos
`CurrentSession`	`NetworkId → Session`	The activ session index per network (`:109`)
`AllocationPerKeyShare`	`NetworkId → Amount`	SRI need per multi key share (`:125`)
`Participants`	`NetworkId → BoundedVec<(Public,u64)>`	The decid validator

Storage	Shape	Purpos
		and their key-share weights (:132)
InSet	(NetworkId,Public) → u64	Key share held by a in-set validator (:145)
TotalAllocatedStake	NetworkId → Amount	Stake counted economi security (:195)
Allocations	(NetworkId,Public) → Amount	Per-valid allocated stake (:2
SortedAllocations	amount-sorted index	O(log n) t N selecti (reverse- lexicogra amount k (:220)
PendingDeallocations	((NetworkId,Public),Session) → Amount	Stake schedule unlock or future session (:305)
Keys	ExternalValidatorSet → KeyPair	The mult key pair s for an external session (:318)

Storage	Shape	Purpos
`PendingSlashReport`	`ExternalNetworkId → …`	The retiri set's key, awaiting slash rep (`:323`)
`SeraiDisabledIndices`	`u32 → Public`	Babe authoriti disabled mid-sess (`:328`)
`SessionBeginBlock`	`NetworkId → BlockNumber`	Block a session began or (`:333`)

Stake, allocations, and key shares

Stake is denominated in SRI. A validator's `Allocation` divided by the network's `AllocationPerKeyShare` gives its number of **key shares** — its weight in the multisig and in consensus. A set is capped at `MAX_KEY_SHARES_PER_SET = 150` shares; validators are taken top-first from `SortedAllocations` until the cap is reached, and if the top validators' shares exceed the cap, `amortize_excess_key_shares` performs a reverse round-robin reduction so the total is exactly 150 (`:388-402` , `:510-511`).

Extrinsics

Call	Index	Origin	Purpose
`set_keys`	0	**unsigned**	Publish an external set's threshold key, MuSig-verified (`:952-991`)

Call	Index	Origin	Purpose
`report_slashes`	1	**unsigned**	Report a retiring external set's slashes — **currently a no-op** (`:993-1019`)
`allocate`	2	signed	Add SRI stake to a network (`:1021-1031`)
`deallocate`	3	signed	Schedule stake to unlock (`:1033-1048`)
`claim_deallocation`	4	signed	Withdraw stake whose unlock session has passed (`:1050+`)

`allocate` / `deallocate` move SRI between the validator and the pallet's own system account via `Coins::transfer_internal` .

> **Flagged gap — external-network slashing is unimplemented.** `report_slashes` verifies the prior set's signature (in `validate_unsigned`) but then **discards the slashes**: the body is `// TODO: Handle slashes` / `let _ = slashes;` and it only emits `SetRetired` (`code/substrate/validator-sets/pallet/src/lib.rs:1007-1016`). So as shipped, a Byzantine external validator reported by its set is *not* actually penalized on-chain. Babe/Grandpa equivocation on the **Serai** set *is* slashed (below); external-network misbehaviour is not.

Fault-tolerance gating (`is_bft`)

`is_bft` returns whether the top validator holds **less than one-third** of the set's key shares (`(top * 3) < key_shares` , `:515`) — i.e. whether the set

survives any single validator turning Byzantine, evaluated *after* the amortization reduction. Allocation changes are gated on preserving this: `increase_allocation` rejects an allocation that would give one validator ≥ 1/3 of `MAX_KEY_SHARES_PER_SET` (`AllocationWouldPreventFaultTolerance`, `:554-555`), and both `AllocationWouldRemoveFaultTolerance` (`:548`) and the economic-security guard (below) constrain allocation/deallocation.

Deallocation scheduling

Deallocations do not release stake immediately. `decrease_allocation` schedules the amount to unlock on a future session (a one-session cooldown, and two extra for the `Serai` set since the next Serai set is already pre-decided), and the stake is only claimable via `claim_deallocation` once that session arrives and the handover completed (`take_deallocatable_amount`, `:773-783`).

Crucially, `TotalAllocatedStake` **intentionally lags allocations**: pending deallocations still count toward economic security and remain slashable until they actually unlock. The code documents this directly — "pending deallocations can still be slashed and therefore still contribute to economic security" (`:905-908`). This is why a removed participant is *not* immediately removed from `TotalAllocatedStake` / `InSet` (`:979-984`): it prevents an economically-secured set from dropping below the threshold the moment someone is removed.

Setting keys — the MuSig gate (`validate_unsigned`)

`set_keys` is an unsigned extrinsic; all its authorization lives in `validate_unsigned` (`:1075-1148`). For a `set_keys` call it:

1. confirms the network has a current session and has **not** already set keys (`Stale` otherwise) (`:1082-1091`);
2. asserts the session equals the latest decided session (`:1095`);

3. de-duplicates `removed_participants` (`:1099-1106`);
4. sums the key shares of the **non-removed** signers and requires `signing_key_shares ≥ 2f + 1` , where `f = all_key_shares - signing_key_shares` (`:1128-1133`) — i.e. a supermajority of shares must be signing;
5. verifies a **MuSig** signature by exactly those signers over `set_keys_message(set, removed_participants, key_pair)` (`:1138-1142`).

`report_slashes` is validated analogously, but against the **prior** set's key taken from `PendingSlashReport` (`:1150-1164`). See Cryptography → DKG for the MuSig construction and The Instruction Interface for the other unsigned extrinsics.

Slashing (Serai set)

Babe/Grandpa equivocation reports route into `slash_serai_validator` (`:889-925`), which zeroes the validator's allocation *and* its current-session pending deallocations, decrements `TotalAllocatedStake` if the validator was in-set, and **burns** the slashed SRI from the pallet's stake account (`Coins::burn` , `:920-924`). `disable_serai_validator` (`:931-947`) additionally marks the validator's Babe index disabled so it can no longer author blocks.

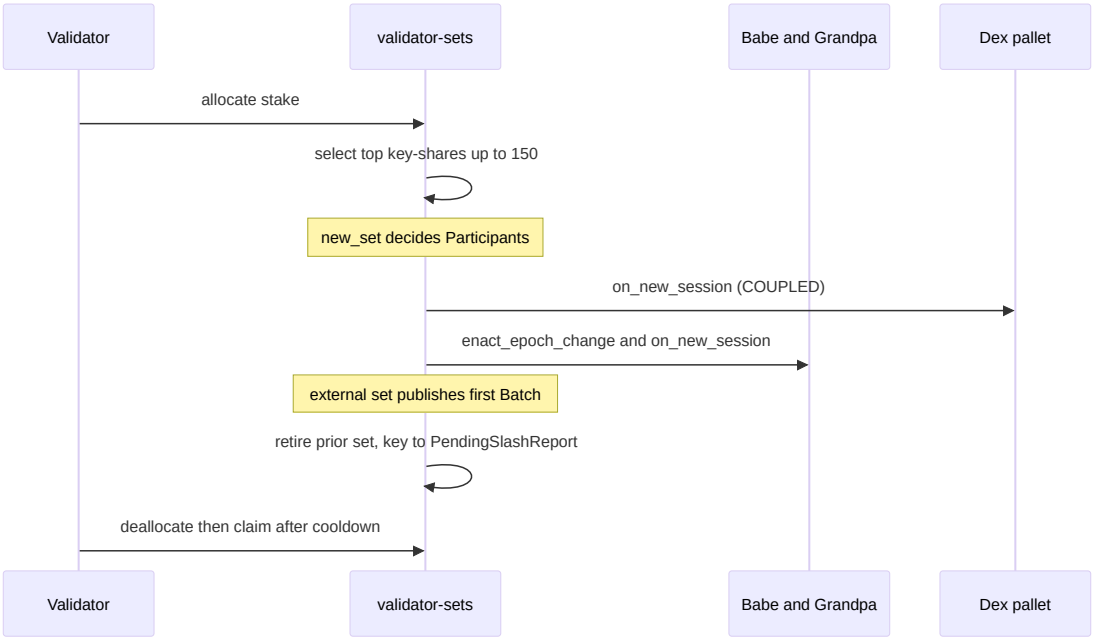
Session rotation

`rotate_session` (`:785-820`) drives the whole handover:

1. retire the current Serai set (`retire_set` , `:741-768`);
2. `new_session` (`:715-728`) spawns the next set for each network whose handover completed (always for `Serai`), and — `[COUPLED]` — calls `Dex::<T>::on_new_session(network)` (`:725`) so the DEX can reset its per-session oracle state (see the boundary chapter §4);

3. hand the new authorities to Babe via `enact_epoch_change` and to Grandpa via `on_new_session` (`:801-819`).

External sets rotate on a different trigger: the retiring set's keys are cleared once the *new* multisig publishes its first `Batch` (see The Instruction Interface and Processor); `retire_set` moves the retiring key into `PendingSlashReport` .



Key invariant

Stake conservation. `TotalAllocatedStake[network]` must equal `Σ Allocations[network,*] + Σ (still-slashable) PendingDeallocations[network,*]` across every allocate / deallocate / slash / rotation path, and the pallet's SRI system-account balance must back it. The deliberate lag (pending deallocations still counted) is central: any path that drops `TotalAllocatedStake` early, double-counts it, or releases stake before its unlock session would break the economic-security guarantee enforced in Economic Security.

The economic-security gate on staking — `required_stake`, `AllowMint`, and the `DeallocationWouldRemoveEconomicSecurity` guard — is documented in the next chapter.

Economic Security

`[COUPLED]` — `code/substrate/validator-sets/pallet/src/lib.rs` (the gate), `code/substrate/economic-security/pallet/src/lib.rs` (the latch), `code/substrate/dex/pallet/src/lib.rs` (the oracle it reads).

This chapter documents the single most important — and most surprising — property of the Serai network: **its custody-safety rule is computed from the DEX application's price oracle.** This is the load-bearing dependency identified in the boundary chapter §4; read that first for the map, then this chapter for the mechanics.

The idea

Serai's multisigs hold real foreign coins (BTC/ETH/DAI/XMR). A validator set that controls a multisig could, if a supermajority colluded, steal everything in it. The defence is **economic**: keep the stake that a colluding supermajority would forfeit larger than the value they could steal, so misbehaving is never profitable. The network's own docs put it as: "the stake required to produce unintended signatures must exceed the value accessible via producing unintended signatures" (`code/docs/economics/index.md`).

To evaluate that, the network needs a **price** for each custodied coin in SRI terms. It gets that price from the DEX's AMM.

The required-stake formula (the code)

`required_stake` values a balance and returns the SRI stake required to secure it (`code/substrate/validator-sets/pallet/src/lib.rs:833-853`):

```
let sri_per_coin = Dex::
<T>::security_oracle_value(balance.coin).unwrap_or(Amount(0)); //
:837
// ... total_coin_value = amount * sri_per_coin / 10^decimals ...
// required stake formula (COIN_VALUE * 1.5) + margin(20%)
let required_stake =
total_coin_value.saturating_mul(3).saturating_div(2); // :851
(×1.5)
required_stake.saturating_add(total_coin_value.saturating_div(5))
// :852  (+×0.2)
```

So the code enforces `required_stake = 1.7 × total_coin_value` (1.5× plus a 0.2× margin). `required_stake_for_network` sums this over the whole supply of each of the network's coins (`:856-863`), reading `Coins::supply(coin)` as the custodied amount.

Spec vs. code — a genuine discrepancy

The specification states the safety bound differently. From `code/spec/protocol/Validator Sets.md:19-23` :

> "This multisig is secure to hold coins valued at up to **33% of the Validator Set's allocated stake**. If the coins exceed that threshold, there's more value in the multisig **and associated liquidity pool** than in the supermajority of allocated stake … it MUST reject newly added coins."

Reconciling the two, as source-of-truth demands (code wins; the gap is flagged):

	Safety bound	Implied stake:value ratio	Value base
Spec (`Validator Sets.md:19-23`)	value ≤ 33% of stake	**stake ≥ 3 × value**	multisig coins **+ liquidity pool**

	Safety bound	Implied stake:value ratio	Value base
Code (`required_stake` , `:850-852`)	stake ≥ 1.7 × value	**stake ≥ 1.7 × value**	coin **supply** only

The on-chain gate is therefore **materially less conservative than the spec**: it permits custodied value up to ≈ stake / 1.7 ≈ **59% of stake**, where the spec's stated ceiling is **33%**. It is also narrower in what it counts — `required_stake_for_network` values only the coin *supply*, not the paired SRI liquidity the spec explicitly says is also at risk. Whether the spec's 3× is over-cautious or the code's 1.7× is under-provisioned is a judgement call, but the two do not agree, and a reader reasoning about custody safety from the spec would over-estimate the protection the code enforces. (Flagged for the audit; see Appendix: Networks & Coins.)

The zero-oracle escape hatch

Note `security_oracle_value(coin).unwrap_or(Amount(0))` at `:837` : when the DEX has **no** observed price for a coin, `sri_per_coin = 0` , so `total_coin_value = 0` , so `required_stake = 0` , so the gate below admits **any** mint. Before the oracle has ratcheted a price, external minting is unconstrained by economic security. The `economic-security` pallet compensates by separately requiring the oracle to be `Some` before it declares a network *secured* (below), but the `AllowMint` gate itself does not.

The mint gate (`AllowMint`)

The runtime wires `coins::Config::AllowMint = ValidatorSets` (`code/substrate/runtime/src/lib.rs:204-206`), and the implementation is (`code/substrate/validator-sets/pallet/src/lib.rs:1185-1196`):

```
fn is_allowed(balance: &ExternalBalance) -> bool {
  let current_required =
Self::required_stake_for_network(balance.coin.network());
  let new_required = current_required +
Self::required_stake(balance);
  let staked =
Self::total_allocated_stake(...).unwrap_or(Amount(0));
  staked.0 >= new_required
}
```

Every `Coins::mint` of an external coin passes through this — including the core bridge-in `InInstruction::Transfer` path (see The Instruction Interface and The Coins Ledger). If minting the new balance would push the network's total required stake above what is actually staked, **the mint is refused** (`MintNotAllowed`). This is the code enforcement of the spec's "MUST reject newly added coins." SRI itself is never gated — only bridged coins (see The Coins Ledger).

The deallocation guard

The same figure protects stake from leaving. `decrease_allocation` recomputes `required_stake_for_network` and rejects a deallocation that would drop `TotalAllocatedStake` below it (`DeallocationWouldRemoveEconomicSecurity` , `code/substrate/validator-sets/pallet/src/lib.rs:604-613`). Combined with the deallocation cooldown (see Validator Sets), this keeps staked value above the custody it secures.

The economic-security latch

The tiny `economic-security` pallet records **when** each network first became economically secure. Its `Config` binds the DEX (`Config: … + DexConfig` , `code/substrate/economic-security/pallet/src/lib.rs:20`), and every block its `on_initialize` (`:38-54`) checks, per external coin:

```
if existing.is_none() &&
   Dex::<T>::security_oracle_value(coin).is_some() &&
   <T as CoinsConfig>::AllowMint::is_allowed(&ExternalBalance {
coin, amount: Amount(1) })
{
  EconomicSecurityBlock::set(coin.network(), Some(n));
  Self::deposit_event(Event::EconomicSecurityReached { network });
}
```

Two DEX dependencies in one condition: an oracle price must exist **and** `AllowMint` (itself DEX-priced) must pass for the smallest unit. The recorded `EconomicSecurityBlock` is consumed by SRI Tokenomics & Emissions and the genesis flow to switch issuance regimes.

> **Flagged — the latch is one-way.** `EconomicSecurityBlock` is only ever *set*, never cleared (`:44-48`): once a network is marked secured, it stays marked even if stake later falls below `required_stake` (e.g. the DEX price rises, or stake is slashed). Downstream logic that treats "reached economic security" as a current fact rather than a historical milestone should be audited against a network that reached security and then regressed.

The oracle it depends on

The value everything above reads is the DEX's `SecurityOracleValue` (`code/substrate/dex/pallet/src/lib.rs:195-199`): *"The price used for evaluating economic security, which is the highest observed median price."* Each block the DEX updates it, but only **upward** — `if median > oracle_value { SecurityOracleValue::set(coin, Some(median)) }` (`code/substrate/dex/pallet/src/lib.rs:481-486`) — so it ratchets to the highest median ever seen (until reset on session rotation via the `on_new_session` call from validator-sets). The `median` is taken over a moving window; the window length is `MEDIAN_PRICE_WINDOW_LENGTH = 2·ARBITRAGE_TIME + 1` with `ARBITRAGE_TIME = 2 hours` (see Constants).

Sampling the *end-of-block* median over a two-hour window is the DEX's deliberate anti-manipulation design: a proposer who spikes the price must sustain it against arbitrage to move the oracle.

The important point for *this* book is not the AMM internals but the direction of the arrow: **the network's security threshold is a function of an application-maintained price.** A weakness in the DEX's median/oracle machinery is, transitively, a weakness in the network's custody guarantee. That is the essence of the coupling in the boundary chapter, and the reason economic security is a network chapter rather than a DEX one.

The Cross-Chain Instruction Interface

`[NETWORK-CORE]` (the pipe) — the payload classification is `[COUPLED]` / `[DEX-APP]` and lives in the boundary chapter.

This is the seam an "application built on Serai" plugs into. A deposit to a Serai multisig on an external chain carries an opaque `data` payload; off-chain that payload is decoded into an **in-instruction**, batched, threshold-signed by the network's validator set, and executed atomically on the Serai chain by `execute_batch`. Value leaves the same way in reverse, as an **out-instruction**.

This chapter documents the *pipe* — the encodings, the batch, and the on-chain validation that makes a batch authoritative. It is unambiguously network infrastructure. **Which instructions the pipe can carry** is where the DEX is wired in; that closed-enum boundary is analysed in Where Serai Ends and the DEX Begins.

1. Shorthand — the wire format

What actually travels in a deposit's `data` is a `Shorthand` (`code/substrate/in-instructions/primitives/src/shorthand.rs:23-37`), a space-optimised envelope:

Variant	Purpose	Status
`Raw(RefundableInInstruction)`	carries a full instruction verbatim	**wired** (`shorthand.rs:50`)
`Swap { origin, coin, minimum, out }`	sugar for a one-shot swap	`todo!()` (`shorthand.rs:51`)

Variant	Purpose	Status
`SwapAndAddLiquidity { origin, minimum, gas, address }`	sugar for swap-and-LP	`todo!()` (`shorthand.rs:52`)

Only `Raw` is implemented today: `TryFrom<Shorthand> for RefundableInInstruction` panics with `todo!()` on both sugar variants (`shorthand.rs:46-55`). So although the format anticipates DEX-specific shorthands, the sole functional path is `Raw` , which wraps whatever `RefundableInInstruction` the sender supplies. The off-chain processor is what `Shorthand::decode` s the deposit bytes — see The Processor — and it does so **without interpreting the instruction's meaning**.

2. `InInstruction`, `Batch`, and the signed message

The decoded payload is a `RefundableInInstruction` (`code/substrate/in-instructions/primitives/src/lib.rs:88-91`): an `InInstruction` plus an optional `origin` external address used for refunds.

The `InInstruction` enum itself (`primitives/src/lib.rs:77-82`) has four variants. Their network/application classification is **canonically defined in the boundary chapter**; summarised here for reference only:

Variant	Site	Class (see boundary chapter)
`Transfer(SeraiAddress)`	`:78`	`[NETWORK-CORE]`
`Dex(DexCall)`	`:79`	`[DEX-APP]`
`GenesisLiquidity(SeraiAddress)`	`:80`	`[DEX-APP]`
`SwapToStakedSRI(SeraiAddress, NetworkId)`	`:81`	`[COUPLED]`

`DexCall` (`primitives/src/lib.rs:65-71`) is itself two DEX operations, `SwapAndAddLiquidity` and `Swap(Balance, OutAddress)`.

Instructions are grouped for submission. An `InInstructionWithBalance` (`:97-100`) pairs an instruction with the `ExternalBalance` credited, and a `Batch` (`:106-111`) is:

```
Batch { network: ExternalNetworkId, id: u32, block: BlockHash,
instructions: Vec<InInstructionWithBalance> }
```

A `SignedBatch` (`:116-126`) attaches an sr25519 `Signature`. The signed message is produced by `batch_message` (`:139-141`):

```
[b"InInstructions-batch".as_ref(), &batch.encode()].concat()
```

i.e. a domain-separated encoding of the whole batch (network, id, block, and every instruction). The signature is the validator set's threshold signature over this message — the *only* thing that makes a batch authoritative on-chain.

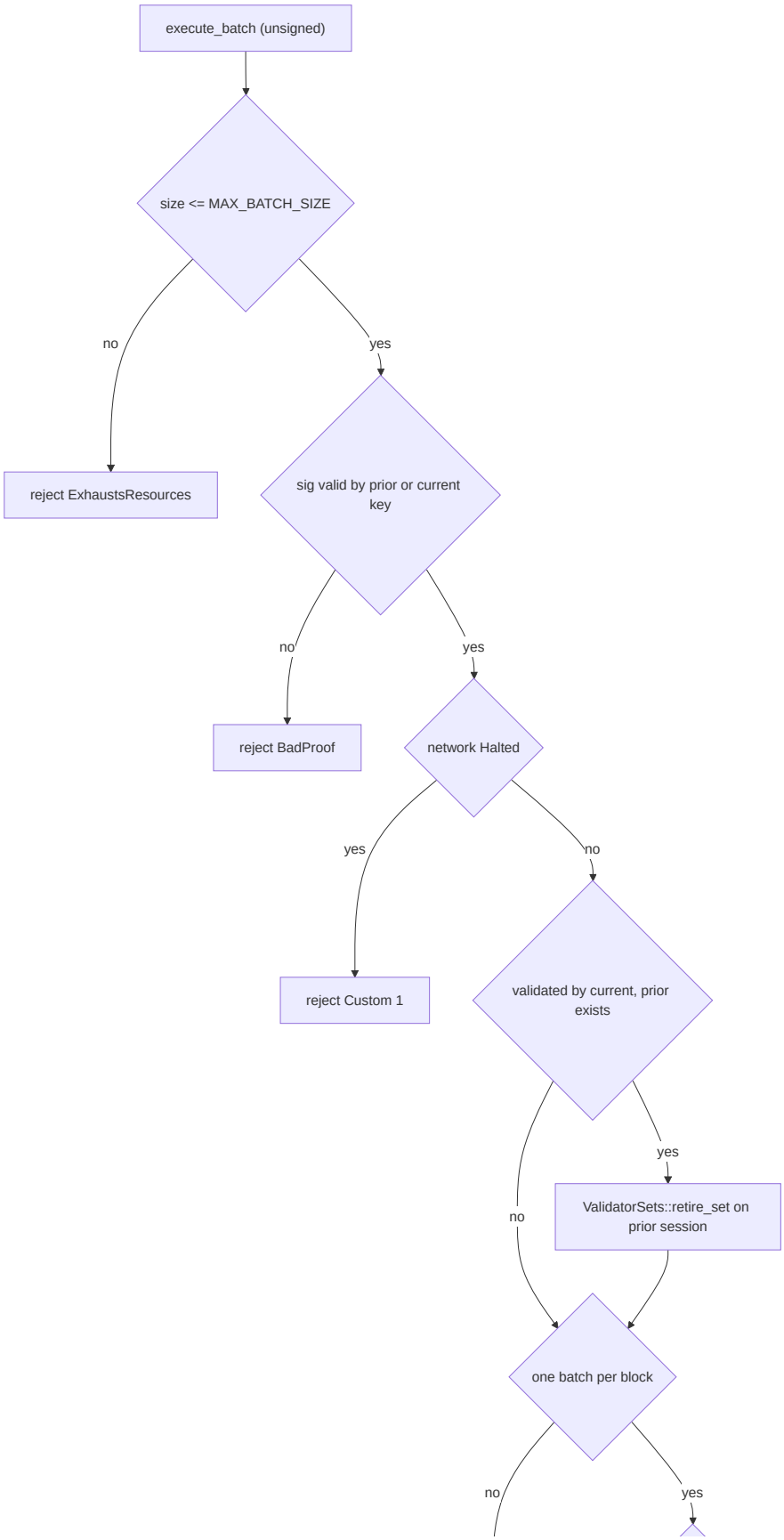
3. `execute_batch` and `validate_unsigned` — the authoritative check

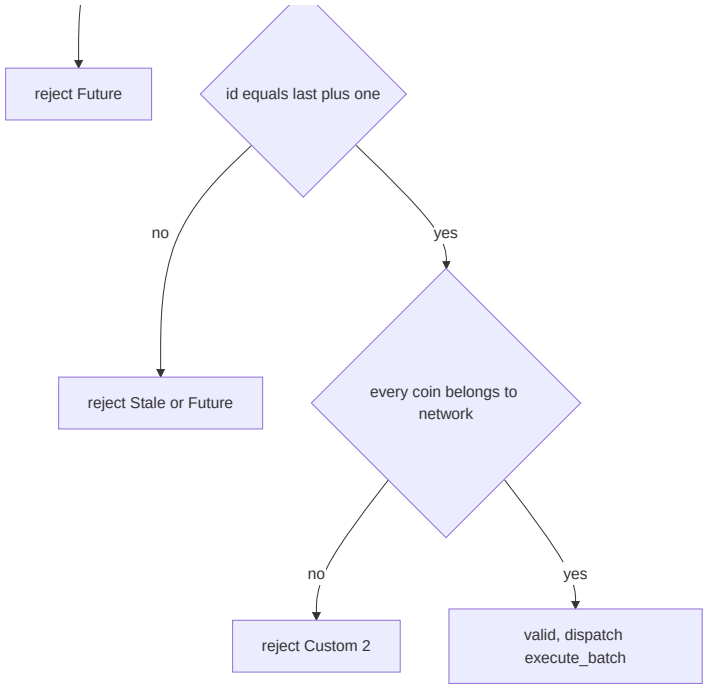
A batch enters the chain through the single extrinsic `execute_batch` (`code/substrate/in-instructions/pallet/src/lib.rs:262-287`), which is **unsigned** (`ensure_none`, `:265`). All authorisation therefore happens in `validate_unsigned` (`:294-379`), which runs before dispatch (a `pre_dispatch` that re-invokes it, `:382-384`). The checks, in order:

1. **Size bound** — `batch.encode().len() > MAX_BATCH_SIZE` (25 000 bytes) → `ExhaustsResources` (`:303-305`, constant at `primitives/src/lib.rs:27`).
2. **Signature** — verified against the network's key via `keys_for_network` (`:239-258`), which returns both the **current** and the **prior** session key. The prior key is checked first, then the current (`:309-323`); failure →

`BadProof`. Accepting either the current or prior key is what makes a **hand-over** between validator sets seamless.

3. **Halt** — if the network is `Halted`, reject with `Custom(1)` (`:325-327`). See Governance for who can halt.
4. **Hand-over retirement (a side effect inside validation).** If the batch validated by the *current* key rather than the prior one (`prior.is_some() && !valid_by_prior`), the pallet calls `ValidatorSets::retire_set` on the prior session (`:329-337`). The first batch the new key signs is treated as the new set accepting responsibility, retiring the old set. This is a state mutation triggered from `validate_unsigned` — worth noting, as it fires on a mempool-reachable path.
5. **One batch per network per Serai block** — `LastBatchBlock` must be `< current_block`, else `Future` (`:340-345`).
6. **Sequential ids** — the next id must be exactly `LastBatch + 1` (or `0`); lower → `Stale`, higher → `Future` (`:347-357`). This gives exactly-once, in-order execution per network.
7. **Network/coin match** — every instruction's `balance.coin.network()` must equal the batch's network, else `Custom(2)` (`:359-371`). The code comments note this can only be hit "if the validator set has turned malicious," in which case the set "should be fully slashed" — but no such slashing is implemented; the comment argues resolution would require a runtime upgrade anyway.


execute_batch (unsigned)
size <= MAX_BATCH_SIZE
no
yes
reject ExhaustsResources
sig valid by prior or current key
no
yes
reject BadProof
network Halted
yes
no
reject Custom 1
validated by current, prior exists
yes
no
ValidatorSets::retire_set on prior session
one batch per block
no
yes



4. `execute()` — per-instruction dispatch

Once a batch is dispatched, `execute_batch` records `LatestNetworkBlock`, emits a `Batch` event with a hash of the instructions, and loops over each instruction calling `execute` (`:276-284`). Crucially, **a failed instruction does not fail the batch**: each `execute` runs in its own transactional layer (`#[frame_support::transactional]`, `:98`), and on error the pallet merely emits `InstructionFailure` and continues (`:277-283`). The batch — and its sequence id — still advances, so the value for a failed instruction is simply never minted.

The `execute` match (`:99-230`) is where instruction meaning is finally assigned:

- `Transfer → Coins::mint(address, balance)` (`:101-103`). The only application-free arm.
- `Dex(SwapAndAddLiquidity)` (`:109-160`) → mint to `IN_INSTRUCTION_EXECUTOR`, swap half for SRI, `Dex::add_liquidity`, refund leftovers. **Slippage is unprotected here:** the internal swap passes `1` as minimum-out (`:123`) and `add_liquidity` passes `1` / `1` minimums (`:136-137`), with a standing TODO acknowledging it (`:141-143`).
- `Dex(Swap)` (`:161-217`) → mint, build the swap `path`, and swap. Unlike the above, this arm **honours the caller's minimum-out** (`out_balance.amount.0`, `:194`). If the destination is external, it

`Coins::burn_with_instruction` s the proceeds to bridge them back out (`:200-216`).

- `GenesisLiquidity` (`:220-223`) → mint to the genesis account, `GenesisLiq::add_coin_liquidity`.
- `SwapToStakedSRI` (`:224-227`) → mint to `POL_ACCOUNT`, `Emissions::swap_to_staked_sri`, which routes value into validator stake (the `[COUPLED]` arm — see Tokenomics and Economic Security).

The unprotected `min = 1` on the `SwapAndAddLiquidity` path is a MEV surface: a Serai block producer who can influence pool prices within a block can sandwich bridged deposits. This is a property of the DEX arm, not the pipe; it is captured as a focus point for the DeFi review.

5. The bridge-out direction

Leaving Serai is the mirror image. Value is burned via `Coins::burn_with_instruction`, producing an `OutInstructionWithBalance` (`code/substrate/coins/primitives/src/lib.rs`) that the processor later plans and threshold-signs into an external-chain withdrawal. The `Dex(Swap)` arm above is one producer of out-instructions (`:204-215`); a plain `Coins::burn_with_instruction` extrinsic is another. See The Coins Ledger for the burn side and The Processor for how an out-instruction becomes a signed transaction.

6. What the spec says, and where it lags

The specification (`code/spec/protocol/In Instructions.md`, `code/spec/integrations/Instructions.md`) describes the same mechanism but likewise hardcodes the DEX: it defines `InInstruction` as `Transfer` and `Dex(Data)`, and its `Swap` / `Add Liquidity` shorthands are entirely DEX-specific. The spec predates the current four-variant enum (it omits `GenesisLiquidity` and `SwapToStakedSRI`), so treat the code as authoritative. The broader point —

that the instruction set is a **closed enum** with only `Transfer` application-free — is developed in the boundary chapter.

Governance & Protocol Changes

`[NETWORK-CORE]`

Serai has “no central authority nor organization nor actors … who can compel new protocol rules. The Serai protocol is as-written” (`code/docs/protocol_changes/index.md:9-12`). The only on-chain governance is a narrow, validator-only signalling mechanism in the `signals` pallet (`code/substrate/signals/pallet/src/lib.rs`) granting validators exactly two powers: **halt an external network** and **retire the protocol** for an upgrade. There is no `sudo` , no council, and no parameter governance — consistent with the runtime exposing no `System` calls to users.

1. Signals and storage

A `SignalId` (`code/substrate/signals/primitives/src/lib.rs:16-19`) is one of:

- `Retirement([u8; 32])` — an opaque 32-byte id naming the consensus rules to migrate to.
- `Halt(ExternalNetworkId)` — halt a specific external network.

State (`pallet/src/lib.rs:68-89`):

Storage	Shape	R
`RegisteredRetirementSignals`	`id -> RegisteredRetirementSignal`	regis retir prop
`Favors`	`(SignalId, NetworkId) -> AccountId -> ()`	per- valid vote favo

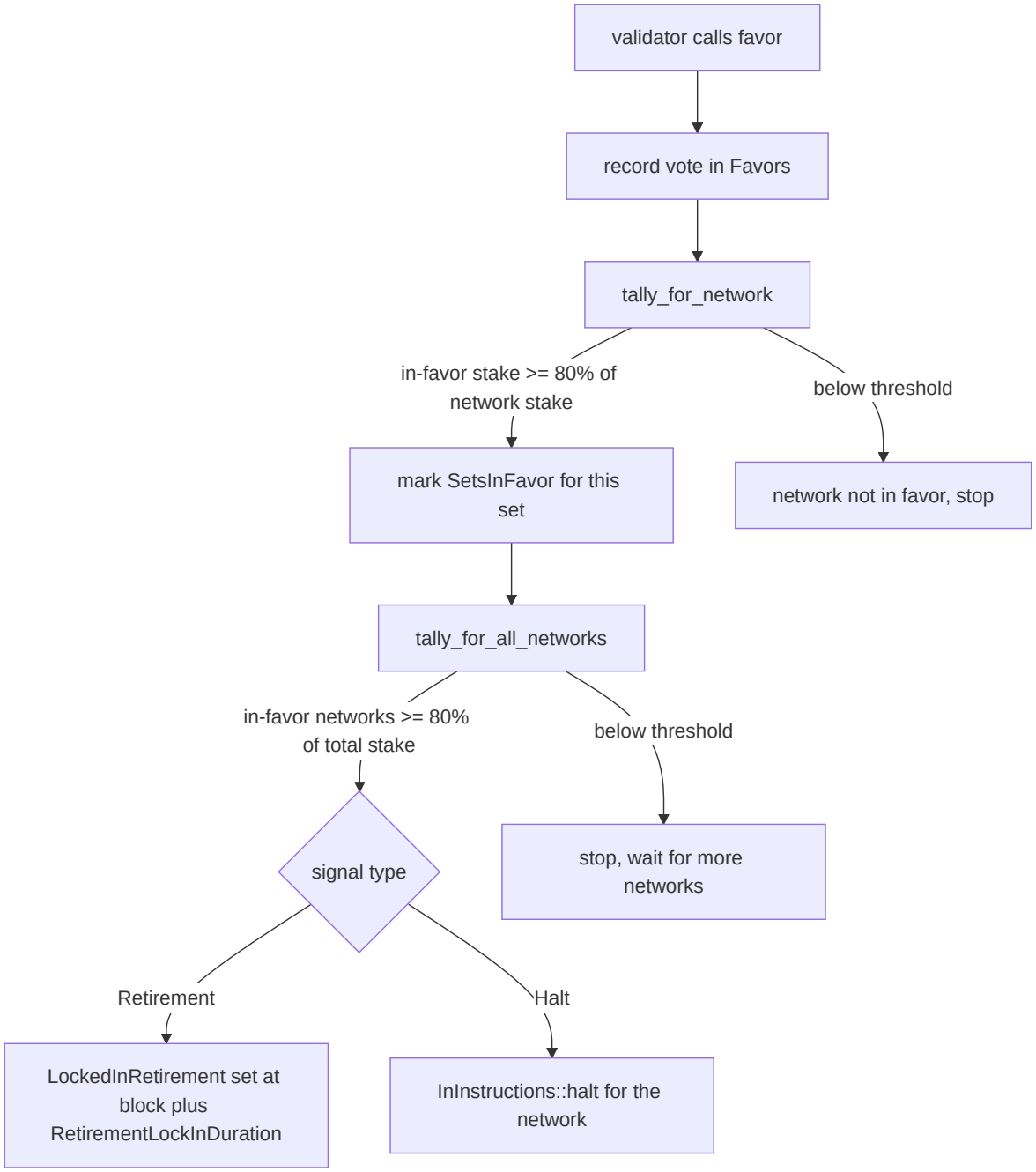
Storage	Shape	R
`SetsInFavor`	`(SignalId, ValidatorSet) -> ()`	whic have cross thres
`LockedInRetirement`	`([u8; 32], BlockNumber)`	the s locke retir + its

A retirement signal is registered by a proposer (`register_retirement_signal` , `:259-291`) and its id is bound to that registrant (`RegisteredRetirementSignal { in_favor_of, registrant, registered_at }` , `:53-58` ; `id()` = `blake2_256("Signal" ++ encode)` , `:60-66`). Binding the id to the registrant prevents an attacker front-running a proposal to become its registrant and later cancelling it (`:270-272`). Only the registrant may `revoke_retirement_signal` (`:295-320`) — and revoking a signal that has already locked in *un-locks* it (`:307-316`), the escape hatch for a fault discovered after lock-in.

2. The favor → tally → lock-in lifecycle

Validators vote with `favor(signal_id, for_network)` (`:324-404`). For a retirement signal the call first checks the signal is registered, not already locked in, and not expired (`registered_at + RetirementValidityDuration` , `:351-355`). In all cases the caller must be a validator of `for_network` — specifically in the **latest decided set** (`in_latest_decided_set` , `:363` , else `NotValidator`). `Halt` signals skip the registration and expiry checks (`:338-340,346-350`).

Each `favor` records the vote and then runs a two-level tally:



- `tally_for_network` (`:150-207`) sums the *current* allocations of in-favor validators and marks `SetsInFavor` once they reach the threshold.
- `tally_for_all_networks` (`:209-229`) sums the stake of all networks whose set is in favor and compares against the same threshold across `NETWORKS` .
- When both clear: a **retirement** locks in, scheduling a halt block `now + RetirementLockInDuration` (`:389-394`); a **halt** takes effect immediately

via `InInstructions::halt(network)` (`:396-398`).

`revoke_favor` (`:409-425`) and `stand_against` (`:432-458`) withdraw or oppose a vote; `stand_against` revokes an existing favor if present, otherwise only emits an event with no state effect (`:454-457`).

3. Thresholds — and the 80%-vs-34% caveat

Both tallies use the **same 80% threshold**: `REQUIREMENT_NUMERATOR = 4`, `REQUIREMENT_DIVISOR = 5` (`:141-144`). A prominent TODO right above it flags that halting *should* use **34%, not 80%** (`:142`). This is a real availability/griefing trade-off, and a genuine spec-vs-code gap: with 80% required to halt, a set of malicious validators controlling just over 20% of stake can *prevent* a halt of a misbehaving network, whereas 34% would let an honest fault-tolerant minority trigger the incident response the Protocol Changes doc describes. Until this TODO is resolved, halting is harder to invoke than the design intends.

The pallet's own comments (`:182-193`) work through the other direction — a validator inflating stake to favor a signal then deallocating — and argue deallocation scheduling (see Validator Sets) largely mitigates it: at most ~50% of a set can be immediately deallocated, so an 80% threshold survives the manoeuvre with a still-BFT majority. Note also that once a set crosses threshold, `SetsInFavor` persists until someone triggers a re-tally, so "a network in favor across any point in its Session is in favor for its entire Session" (`:176-198`) — a staleness property worth keeping in mind.

4. Retirement halts the chain by deliberate panic

Retirement is enforced by the one place in the entire codebase where a `panic!` is *intended* behaviour rather than a bug. `on_initialize` checks whether the current block is the locked-in halt block and, if so, panics to stop the block executing and thereby halt the chain (`:463-475`):

```rust
if block_number == current_number {
    panic!("locked-in signal {} has been set for too long", …);
}
```

Every other service in Serai installs a panic hook that turns any panic into `process::exit(1)` (a liveness hazard discussed in the off-chain overview); here the halting panic is the feature. Nodes are then expected to individually hard-fork to new consensus rules that remove the halt (`code/docs/protocol_changes/index.md:33-36`).

5. Halting an external network

`Halt(network)` calls `InInstructions::halt` (`:396-398`), which sets the network `Halted` and thereafter causes `execute_batch`'s `validate_unsigned` to reject that network's batches with `Custom(1)`. Per the protocol doc, halting a set "prevents further publication of `Batch`s … and preventing validators from unstaking … This effectively burns the malicious validators' stake" (`code/docs/protocol_changes/index.md:16-25`), because unstaking requires the successor set to accept responsibility, which requires batch publication.

6. Protocol philosophy — the canonical chain

Governance is deliberately minimal because Serai's design treats the protocol as fixed rules plus a soft, non-binding policy on how to interpret the external world. The Canonical Chain policy (`code/spec/policy/Canonical Chain.md`) states Serai honors only the chain it has **finalized** (`:13-18`), treats a hard fork or rollback that "breaks Serai's integrity" as unsupported (`:28-42`), and — importantly for the network-vs-application boundary — commits to destroying application state under duress: "if Serai lists a token which has a by-governance blacklist function, and is blacklisted without appeal, Serai should destroy all associated `sriXYZ` and cease operations" (`:63-65`). On a bug losing a fraction of coins, losses are amortized proportionally across all holders and swaps are halted until users can withdraw (`:67-72`). Across any hard fork, the `sriXYZ`

balances survive unless the new rules change them
(`code/docs/protocol_changes/index.md:38-44`).

Genesis enforces one structural invariant tying the two durations together: `RetirementValidityDuration < RetirementLockInDuration` (`:44-46`), so a lock-in period automatically outlasts (and thus invalidates) competing retirement signals.

SRI Tokenomics & Emissions

SRI is Serai's native coin — the gas, staking, and settlement asset of the network. This chapter documents where SRI comes from (genesis and emissions), how it is distributed, and how much of that monetary policy is **pure network** versus **coupled to the DEX**. The coin itself and the fee burn are `[NETWORK-CORE]`; the emission *distribution* is `[COUPLED]`, because after economic security is reached it is driven by DEX swap volume and DEX prices. The canonical boundary is in the boundary chapter; this chapter fills in the numbers.

SRI, the native coin — [NETWORK-CORE]

SRI is the `Coin::Serai` variant of the ledger's coin type. It is distinguished from every bridged coin at the type level and treated as privileged throughout the runtime (see The Coins Ledger):

- Name `"Serai"`, symbol `"SRI"`, **8 decimals** (`code/substrate/primitives/src/networks.rs:369-388`).
- `Coin::native() == Coin::Serai` and `is_native()` matches only SRI (`code/substrate/primitives/src/networks.rs:358-360,390-392`).
- Unlike bridged coins, SRI is minted freely by the runtime (emissions, rewards, genesis) — its mint is **not** gated by the `AllowMint` economic-security check that external coins face (see Economic Security).

A fair launch

SRI has no pre-distribution. Per the project's own statement, "SRI will have no ICO, no IEO, no presale, no developers' tax/fund, and no airdrop for out-of-mainnet activity … All distributions of SRI will happen on-chain per the protocols' defined rules, based on on-chain activity" (`code/docs/index.md:21-32`). Testnet participation, community membership, and GitHub contribution

are explicitly *not* rewarded. Every SRI in existence therefore originates from one of the on-chain mechanisms below.

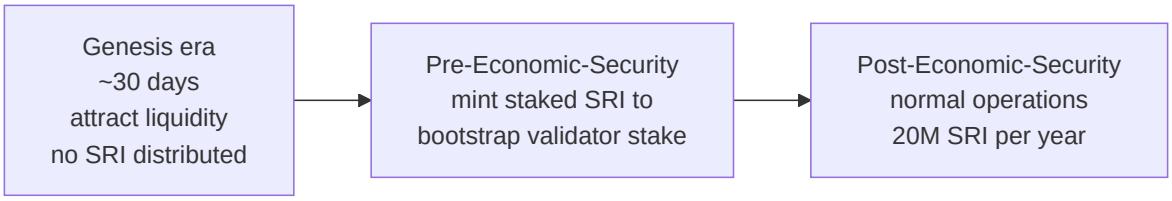
Supply and constants

Constant	Value	
`GENESIS_SRI`	100,000,000 SRI (`100_000_000 * 10^8`)	`code/subst`
`GENESIS_SRI_TRICKLE_FEED`	6 months of blocks	`code/subst`
`INITIAL_REWARD_PER_BLOCK`	100,000 SRI/day (for the initial period)	`code/subst 11`
`REWARD_PER_BLOCK`	20,000,000 SRI/year	`code/subst 14`
`SERAI_VALIDATORS_DESIRED_PERCENTAGE`	20 (% of stake targeted for the Serai network)	`code/subst 17`
`SECURE_BY`	1 year (target block for economic security)	`code/subst 26`
`POL_ACCOUNT`	protocol-owned-liquidity system account	`code/subst 8`
`TARGET_BLOCK_TIME`	6 seconds	`code/subst`

Time units are all block counts derived from `TARGET_BLOCK_TIME` : a “month” is defined as exactly 30 days and a “year” as 12 such months (360 days, ~1.5% short of a calendar year) — `code/substrate/primitives/src/constants.rs:13-16` . Cite these definitions when reasoning about any “60 days”/“6 months” figure below; they are approximations by construction.

The three economic eras

Serai’s economics change across three eras (`code/docs/economics/index.md`). The switch is driven by whether every external network has reached economic security (`pre_ec_security()` , `code/substrate/emissions/pallet/src/lib.rs:321-329`).



- **Genesis (30 days).** The goal is to attract the liquidity needed to enable swaps; anyone may add liquidity, and **no SRI is distributed** until the era ends (`code/docs/economics/index.md:13-18`). The genesis period ends at block `MONTHS` (`code/substrate/genesis-liquidity/pallet/src/lib.rs:95`).
- **Pre-Economic Security.** With liquidity and swaps live, the network mints freshly staked SRI to would-be validators who swap external coins for stake, driving toward the point where stake exceeds the value it secures (`code/docs/economics/index.md:22-39`). See Economic Security for the definition and the stake threshold.
- **Post-Economic Security.** A steady state (barring upgrades), issuing `REWARD_PER_BLOCK` (`code/docs/economics/index.md:41-46`).

Emissions — [COUPLED]

Emissions are computed in `Emissions::on_initialize`, which only acts on a Serai session change after genesis (`code/substrate/emissions/pallet/src/lib.rs:105-126`). The reward for the epoch and its split differ by era.

Pre-Economic-Security issuance

The total epoch reward is either the fixed initial-period rate or a distance-to-security rate:

- During the **initial period** (the first `2 * MONTHS` ≈ 60 days after genesis, `code/substrate/emissions/pallet/src/lib.rs:309-319`), reward is `block_count * INITIAL_REWARD_PER_BLOCK` (`:158-160`).
- After that, reward is `(Σ distance-to-required-stake) / blocks_until(SECURE_BY)` per block (`:162-165`).

The per-network split is by each network's **distance to economic security**, where `required_stake_for_network` is itself DEX-priced (`code/substrate/emissions/pallet/src/lib.rs:140`; see Economic Security). The Serai network is carved out a `SERAI_VALIDATORS_DESIRED_PERCENTAGE` (20%) share (`:152-156`). This makes even the *pre*-security schedule depend on the DEX oracle transitively, through `required_stake_for_network`.

Post-Economic-Security issuance — DEX-driven

Total reward is `block_count * REWARD_PER_BLOCK` (`code/substrate/emissions/pallet/src/lib.rs:169`), split:

- **20% to the Serai network**, i.e. `reward / 5` (`:222-224`).
- **80% across external networks by DEX swap volume.** The pallet reads `Dex::swap_volume(coin)` per external coin, diffs it against `LastSwapVolume` to get the epoch's volume, and apportions the 80% by each network's share of total volume (`:191-235`, volume read at `:194`).

> **This is the monetary policy's hard dependency on the DEX application** — with no swaps, `total_volume == 0` and the code has an acknowledged open question about what to do (`:226` , returns 0).

Within an external network, the reward is further split between validators and the pool by an unused-capacity ratio, and the **pool share is minted into the DEX pool account** `Dex::get_pool_account(coin)` (`code/substrate/emissions/pallet/src/lib.rs:255-283` , mint at `:274-276`). Validator rewards are minted then staked via `ValidatorSets::distribute_block_rewards` (`:352-353`), which credits the reward as an allocation **bypassing the normal minimum-allocation gate** (it is a protocol reward, not a user allocation).

`swap_to_staked_sri` — an application swap that becomes stake — [COUPLED]

The `SwapToStakedSRI` in-instruction (see Instructions) lands here. Only permitted pre-economic-security (`code/substrate/emissions/pallet/src/lib.rs:365-382`), it: swaps half the deposit for SRI to form protocol-owned liquidity (`Dex::swap_exact_tokens_for_tokens` , min-out `1` , `:388`), adds liquidity (`Dex::add_liquidity` , `:400`), values the deposit at the **last block's spot price** (`Dex::spot_price_for_block` , `:412` — a single-block spot, not the manipulation-resistant median used for security; flagged with a "may panic" TODO at `:414-416`), mints that much SRI to the user, and stakes it via `ValidatorSets::allocate` (`:419-426`). It is the most direct expression of the coupling: an application swap converted straight into network stake.

Genesis liquidity bootstrap — [DEX-APP], network-relevant

The one-time bootstrap lives in the `genesis-liquidity` pallet, which is DEX-application code but matters here because it is where the 100M `GENESIS_SRI`

enters circulation. Briefly (`code/substrate/genesis-liquidity/pallet/src/lib.rs`):

- Users deposit external coins during genesis (via the `GenesisLiquidity` in-instruction), receiving genesis-LP shares (`INITIAL_GENESIS_LP_SHARES = 10_000` , `code/substrate/genesis-liquidity/primitives/src/lib.rs:20` ; `add_coin_liquidity` at `:186-218`).
- At the end of genesis, `on_initialize` mints `GENESIS_SRI` and distributes it across pools in proportion to each pool's value, with the last pool taking the remainder and an assertion that the full `GENESIS_SRI` was distributed and the genesis account drained (`:90-173` , mint `:104-105` , `assert_eq! :168,173`).
- Pool values come from `oraclize_values` , an **unsigned, MuSig-verified** extrinsic where **BTC is hardcoded to `10^decimals`** and Monero/Ether/DAI values are supplied by the validator set (`:399-410`). This is a validator-attested oracle input — see Oracle & cross-chain trust for how such values are agreed.
- Post-genesis withdrawals return the SRI portion on a **6-month linear trickle-burn** (`GENESIS_SRI_TRICKLE_FEED` , `:316-321`), gated on `blocks_since_ec_security` (`:231-...`).

For the AMM mechanics these pools feed, and the price oracle they produce, see the boundary chapter; this book does not document the DEX's internal math.

Fee burn — [NETWORK-CORE]

Transaction fees are paid in SRI (the runtime wires `OnChargeTransaction = Coins`) and are **deflationary**: `Coins::on_initialize` burns the *entire* fee-account balance at the start of every block (`code/substrate/coins/pallet/src/lib.rs:115-124`). The burn uses `.unwrap()` , and the code notes that a failure here would halt the runtime because it runs every block (`:119-121`) — a network-liveness consideration, not a DEX one. See The Coins Ledger.

What is network vs coupled, summarized

- **Pure network monetary policy:** the existence and privileged minting of SRI, the fixed genesis supply, the deflationary fee burn, and the *intent* of the emission schedule (bootstrap stake, then a fixed yearly issuance).
- **DEX-coupled:** the *distribution* of emissions once economic security is reached (split by `Dex::swap_volume`, paid into DEX pool accounts), the pre-security split (via DEX-priced `required_stake_for_network`), and `swap_to_staked_sri`'s spot-priced conversion. None of these can be evaluated without the DEX pallet present and active — the reason `Emissions` is tagged `[COUPLED]` and not `[NETWORK-CORE]`.

Overview & the Message Bus

[NETWORK-CORE] — Everything in Part III runs *off* the Serai blockchain, on each validator's own machines. It is the part of Serai most people never see: the daemons that watch Bitcoin, Ethereum and Monero, hold the threshold-multisig key shares, reach agreement with the other validators, and turn on-chain decisions into signed external-chain transactions. Critically for this book's thesis, **none of this layer mentions the DEX.** Its abstractions stop at "an external network" and "a batch of instructions"; application meaning is assigned only later, on-chain (see the boundary chapter §5).

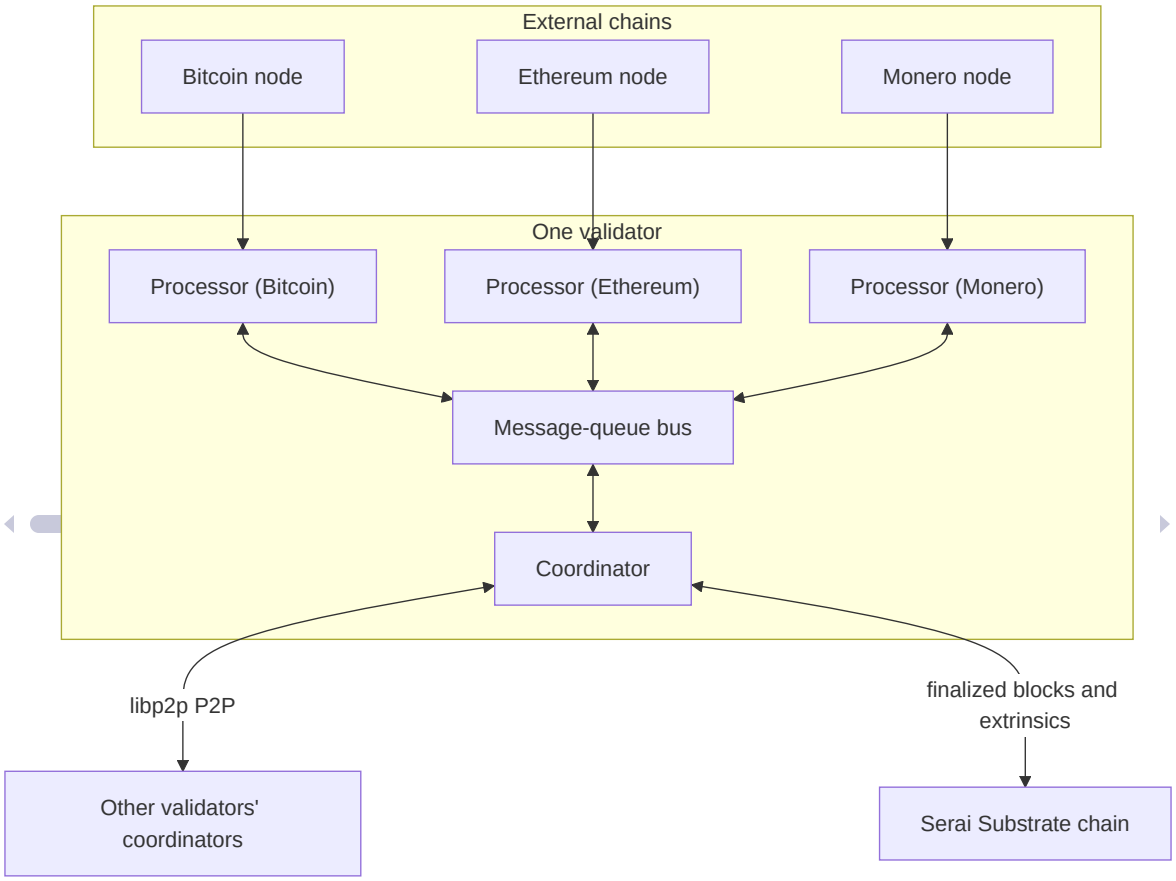
The four components

Component	One per…	Responsibility	Crate
Processor	external network, per validator	Scan a chain for deposits, build `Batch`es, threshold-sign withdrawals; holds the key shares	`code/processor`
Coordinator	validator	Orchestrate DKG/signing, run per-set **Tributary** BFT chains, talk P2P to other validators, publish to Serai	`code/coordinat`
Message-queue	deployment	An ordered, persistent, authenticated bus carrying messages between a	`code/message-queue`

Component	One per…	Responsibility	Crate
		coordinator and its processors	
Networks	—	Chain-integration libraries (Bitcoin/Ethereum) the processor builds on	`code/networks`

Their wiring, from the repo's own component taxonomy (`code/README.md:34-40`):

> "*processor*: A generic chain processor to process data for Serai and process events from Serai… *coordinator*: A service to manage processors and communicate over a P2P network with other validators."



Each processor speaks **only** to the coordinator, and only through the message-queue; processors never talk to each other or to the Serai chain directly. The coordinator is the single hub that bridges the three trust domains — the Serai chain, the local processors, and the peer validators.

The message-queue

The message-queue (`code/message-queue/src`) is a small, consistency-focused server: an ordered, persistent, authenticated mailbox so a coordinator and its processors can exchange messages reliably across restarts and offline peers. It runs a single-threaded Tokio runtime and, like every service here, treats any task panic as fatal (below).

Authentication model

Every service — `Coordinator` and `Processor(Bitcoin|Ethereum|Monero)` — registers a Ristretto public key from its environment. Authenticity rests on **per-message signatures, not the transport**:

- `queue_message` verifies a Schnorr signature over a transcript binding `from / from_key / to / intent / msg / nonce` before accepting a message (`code/message-queue/src/main.rs:60-74` , challenge in `code/message-queue/src/messages.rs:37`).
- `ack_message` verifies a signature by the *recipient's* key over `to / from / id / nonce` (`code/message-queue/src/main.rs:135` , challenge in `messages.rs:58`).
- A hard **topology invariant** is asserted: every message is coordinator ↔ processor — `assert!(matches!(meta.from, Coordinator) ^ matches!(meta.to, Coordinator))` (`code/message-queue/src/main.rs:74`).
- The `Next` (read) RPC is **intentionally unauthenticated** — the server holds no secrets and this avoids per-read challenge/response. The recipient is expected to re-verify the forwarded signature itself, though

the client still carries `// TODO: Verify the sender's signature` (`code/message-queue/src/client.rs:202`).

Ordering, dedup, and acking

Messages are held in a per-`(from,to)` queue. `queue_message` assigns a sequential `id` and dedups by intent: a repeated `(from, to, intent)` is dropped, making sends idempotent (`code/message-queue/src/main.rs:76-94`). `get_next_message` returns `last_acknowledged + 1` (`:120-123`), and `ack_message` advances `last_acknowledged` — with an open `// TODO: Check only a proper message is being acked` (`:143`), i.e. no bound check that the acked id is in range, a correctness gap worth noting for anyone reasoning about message loss.

Transport & the panic-driven DoS surface

The server binds `0.0.0.0:2287` (`code/message-queue/src/main.rs:230`) with the transport itself unauthenticated (`// TODO: Add a magic value with a key … to make this authed`, `:234`). Because sender authenticity is enforced only by the per-message Schnorr signature, and because malformed input is handled with `.unwrap()` / `assert!` / map-indexing on attacker-supplied fields, a malformed packet from anyone who can reach the port can panic the process.

This is an instance of a **cross-cutting theme across all of Part III**: every service installs a panic hook that calls `std::process::exit(1)` — for the message-queue at `code/message-queue/src/main.rs:155-160`, and identically in the processor (`code/processor/src/main.rs:704-709`) and the coordinator. A single panic in any Tokio task therefore takes down the whole service. Any attacker-reachable `unwrap!` / `panic!` / `assert!` on network, RPC, or peer input is thus a **remote liveness / denial-of-service** concern, not merely a local bug. Keep this in mind through every following chapter.

The Processor

[NETWORK-CORE] — The processor is the per-external-network daemon that gives Serai its hands on a foreign chain. Running one instance per network per validator, it does two jobs:

1. **Scan inward** — watch an external chain (Bitcoin / Ethereum / Monero) for deposits to the multisig, decode each into an `InInstruction`, and package them into `Batch`es for the coordinator.
2. **Sign outward** — take Serai's decisions (withdrawals, change consolidation, multisig-rotation forwards, refunds) and construct + FROST-threshold-sign the external-chain transactions that carry them out.

It holds the network's **key shares**, is driven entirely by the message-queue to and from the coordinator, and never talks to the Serai chain directly — Serai state reaches it only as `CoordinatorMessage::Substrate(...)`. Like the rest of the off-chain stack it is **DEX-agnostic**: it produces and consumes generic instructions and never interprets swap semantics (see the boundary chapter §5).

The generalized `Network` trait — the add-a-chain seam

The one real extension point in the whole off-chain stack is the `Network` trait (`code/processor/src/networks/mod.rs:243`). It is the complete abstraction of "an external chain":

```
pub trait Network: 'static + Send + Sync + Clone + PartialEq + Debug
{
    type Curve; type Transaction; type Block; type Output;
    type SignableTransaction; type Eventuality; type
TransactionMachine; type Scheduler; type Address;
    const NETWORK: ExternalNetworkId;   // :283
    const ID: &'static str;             // :285
    const CONFIRMATIONS: usize;         // :289
    const MAX_OUTPUTS: usize;           // :293
    const DUST: u64;                    // :301
    const COST_TO_AGGREGATE: u64;       // :304
    // get_latest_block_number / get_block / get_outputs /
signable_transaction / attempt_sign /
    // publish_completion / confirm_completion / address derivations
…
}
```

To add a chain, a developer implements `Network` (plus its associated `Output` / `Transaction` / `Eventuality` / `SignableTransaction` / `Scheduler`) — exactly as `bitcoin.rs`, `ethereum.rs` and `monero.rs` do, each behind a cargo feature. But note the boundary this seam does **not** cross: the network *identity space* (`ExternalNetworkId`) is a closed enum in the on-chain primitives, so a new chain is still a runtime fork, not a plugin. See External Network Integrations.

Main loop and the two ownership domains

`run()` boots state, then loops over two event sources under one DB transaction committed per iteration: coordinator messages (`coordinator.recv()`) and autonomous scanner events. A coordinator message is acked only *after* its effects commit, and strict ordering is asserted (`assert_eq!(msg.id, last + 1)`).

The processor's mutable state is split into two carefully-separated domains (`code/processor/src/main.rs:76-137`):

Domain	Holds	Driven by
`TributaryMutable`	`key_gen`, `signers`, `batch_signer`, `cosigner`, `slash_report_signer`	Tributary / coordinator messages
`SubstrateMutable = MultisigManager<D, N>`	the `Scanner` + per-key `Scheduler`s	Substrate-acked blocks + the autonomous scanner

The safety argument (`main.rs:117-137`) is that Tributary- and Substrate-triggered messages never need a simultaneous mutable borrow of the same data — an invariant worth scrutiny, since it is the processor's core concurrency claim.

Entropy & custody. All key material derives from a single 32-byte `ENTROPY` env var seeding a `RecommendedTranscript` (`main.rs` boot, length-checked with `panic!("entropy isn't the right length")` at `:491`); a `ZeroizingAlloc` global allocator wipes freed memory. Key shares live at rest in RocksDB/ParityDB protected only by `Zeroizing`, **not encrypted at rest**. A panic in any task exits the whole process via the panic hook (`main.rs:704-709`) — the liveness theme from the overview.

Scanning inward

The `Scanner` (`code/processor/src/multisigs/scanner.rs`) runs autonomously in a spawned task and treats `CONFIRMATIONS`-depth as finality:

- It scans only up to `latest - (CONFIRMATIONS - 1)` (`:481`).
- A reorg past a finalized block is unrecoverable: `panic!("reorg'd from finalized …")` (`:523,529`).
- It refuses to scan more than `need_ack + CONFIRMATIONS` ahead of an un-acked `Batch` (`need_ack` queue at `:220`; `limit = needing_ack + N::CONFIRMATIONS` at `:465,502`) — the **rotation-safety throttle** that

bounds how far ahead of Serai's acknowledgement the processor may run.
- Outputs below `DUST` are dropped; double-scan protection ends in `panic!("scanned an output multiple times")` (`:637`). The scanner guarantees at-least-once delivery and may duplicate.

Deposit crediting

`instruction_from_output` (`code/processor/src/multisigs/mod.rs:40-91`) turns a raw output into a credited instruction. It is deliberately application-blind: it `Shorthand::decode`s the output's opaque `data` (`:66`), converts to a `RefundableInInstruction` (`:73`), and emits an `InInstructionWithBalance` (`:91`). Crucially it deducts `2 * COST_TO_AGGREGATE` from the credited amount to blunt miner/economic griefing — `balance.amount.0 -= 2 * N::COST_TO_AGGREGATE;` (`:87`). That this is a plain `-=` on an attacker-influenceable amount is exactly the kind of underflow an audit should chase (a lying external-chain RPC is a first-class threat here — see the untrusted-RPC focus point and External Network Integrations).

Multisig rotation

When a validator set hands over to its successor, the processor migrates custody across a **6-step rotation** (`RotationStep`, `current_rotation_step`, in `code/processor/src/multisigs/mod.rs`): `UseExisting → NewAsChange → ForwardFromExisting → ClosingExisting`, on time-based windows derived from the new key's activation block. `filter_outputs_due_to_closing` carries a long comment documenting an Eventuality/reorg race and the heuristic used in place of provably-correct output attribution — prime audit territory, and the reason the scanner throttle above exists.

Planning & fee amortization — funds conservation

Outbound value is planned by the schedulers (`code/processor/src/multisigs/scheduler/utxo.rs` for UTXO chains, `.../smart_contract.rs` for Ethereum) and finalized by `prepare_send` (`code/processor/src/networks/mod.rs:424`). This is where funds-conservation math lives:

- `prepare_send` computes the needed fee and **amortizes it across payments** (`:458-466`), drops sub-`DUST` payments, and converts un-createable change into `operating_costs` (`:239,452`) — a shortfall later payments recoup only if a change output exists.
- The `assert!` guards (e.g. `:431`) and the `saturating_sub`s in the amortization loop are the key invariants; an off-by-one or `u64` underflow here mis-credits or loses funds. A `Plan.id()` (a transcript over key + inputs + payments + change) is the sole authorization for spending, and also the FROST signing message / eventuality identity.

Signing outward

Two signing families run, both FROST but over different curves:

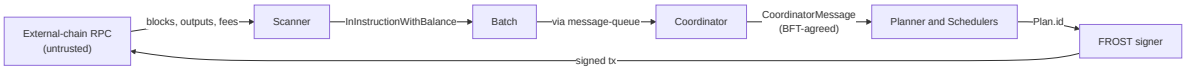
- **Network transactions** — `code/processor/src/signer.rs` runs FROST for withdrawals/rotation transactions. On a reboot mid-sign it **deliberately aborts and waits for a fresh attempt rather than reuse the preprocess RNG** (reusing a preprocess would leak the key share — `signer.rs:171` holds the per-attempt `preprocessing` map; the reboot handling is at `:227-231`). A `rebroadcast_task` (`:190`) re-publishes unconfirmed completions periodically.
- **Substrate messages** — `batch_signer.rs`, `cosigner.rs`, `slash_report_signer.rs` each use `AlgorithmMachine<Ristretto, Schnorrkel>` (domain `b"substrate"`) to sign `Batch`es, block cosigns, and slash reports for on-chain submission.

See FROST Threshold Signing for the preprocess/nonce safety these rely on.

Key generation & custody

`code/processor/src/key_gen.rs` runs PedPoP DKG simultaneously for two curves — Ristretto (the Substrate/batch key) and the network's own `Curve` — with a deterministic RNG derived from `ENTROPY` plus a per-`(session, network, attempt)` transcript context, so a DKG can resume across reboots. Generated keys are persisted but only promoted to active use on the Substrate `ConfirmKeyPair` message, and `N::tweak_keys` applies chain-specific adjustments on load (e.g. BIP-340 even-Y for Bitcoin). See Distributed Key Generation.

Trust boundaries



1. **External-chain RPC** — fully untrusted node output (block contents, outputs, `presumed_origin`, fee estimates). Mitigated by `CONFIRMATIONS` depth, reorg panics, `DUST` filtering, the `2*COST_TO_AGGREGATE` deduction, and data-length limits — but the processor trusts the RPC for finalized history, so a compromised node is a first-class threat.
2. **Coordinator messages** — semi-trusted (they represent the Tributary/Substrate BFT consensus). Many handlers `panic!` / `expect` on unexpected content (`main.rs:245,248,262`), so a faulty coordinator can halt the processor; per-signer FROST-share validation defends against individual malicious signers.
3. **Serai instructions** — reach the processor only as `substrate::CoordinatorMessage` and are trusted as Serai consensus.

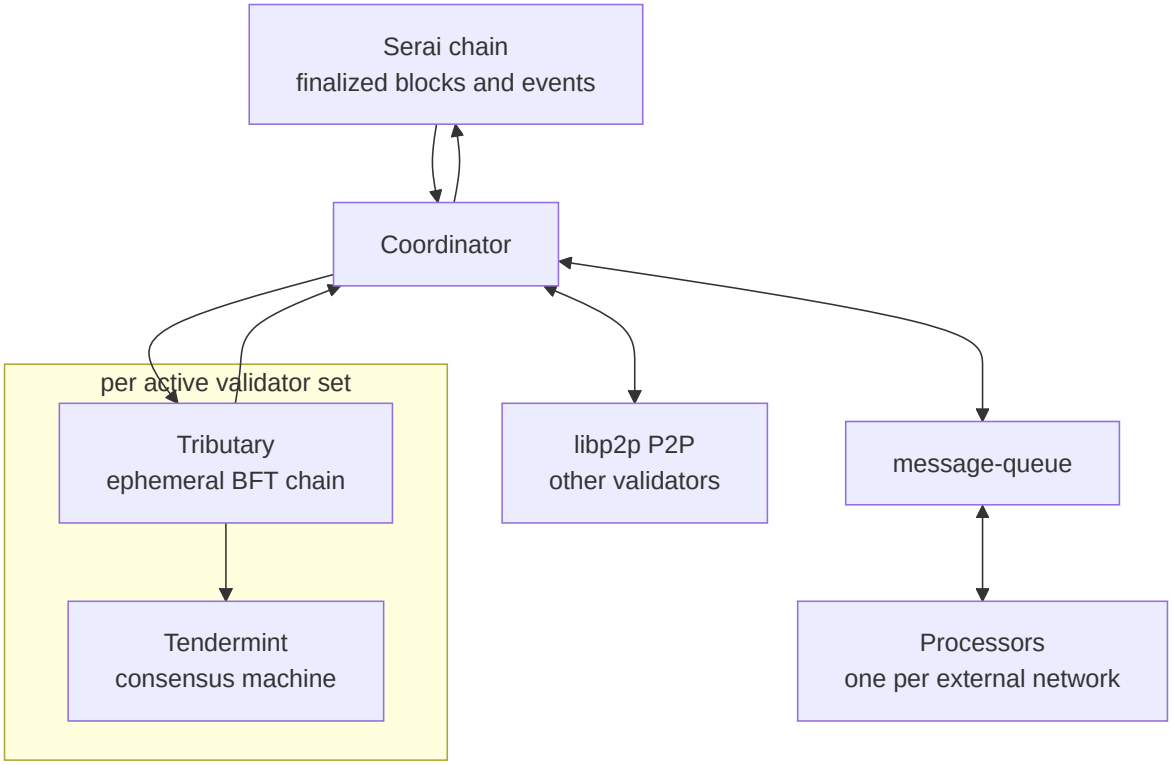
The Coordinator

`[NETWORK-CORE]` — the coordinator is application-agnostic; it never mentions the DEX (see the boundary chapter, §5).

The coordinator is the per-validator orchestration service (`code/coordinator/src/`). It sits between three trust domains and reconciles them:

1. the **Serai chain** — it reads finalized blocks/events and publishes set-keys, batches, and slash-reports;
2. the local **processors** — one per external network, reached over the message-queue bus;
3. **other validators** — reached over a libp2p peer-to-peer network.

Its central job is agreement. Threshold operations (distributed key generation, FROST signing) need every validator in a set to act on the *same* inputs in the *same* order. To get that under Byzantine conditions, the coordinator runs — for each active validator set — a dedicated, ephemeral BFT blockchain called a **Tributary**, driven by a from-scratch **Tendermint** state machine. Agreement reached on a Tributary is then relayed to the processors as ordered instructions.



Service wiring

`run()` in `code/coordinator/src/main.rs` spawns the long-lived tasks: a substrate scanner (`substrate::scan_task`), a Tributary block scanner (`tributary::scanner::scan_tributaries_task`), the P2P handler (`p2p::handle_p2p_task`), a heartbeat/syncer (`p2p::heartbeat_tributaries_task`), a cosign evaluator (`CosignEvaluator::new`), and the processor-message loop (`handle_processors` → `handle_processor_message`).

A single fact dominates the coordinator's threat model: a **global panic hook turns any task panic into `process::exit(1)`** (`code/coordinator/src/main.rs`). Any reachable `unwrap()` / `panic!` / `assert!` on an attacker-influenced path is therefore a liveness/denial-of-service concern, not a recoverable error. This pattern recurs across every Serai service; it is called out again in the off-chain overview and processor chapters.

`handle_processor_message` (`code/coordinator/src/main.rs`) is the large dispatcher translating each `ProcessorMessage` (KeyGen / Sign / Coordinator / Substrate) into Tributary transactions, and also directly publishing `SignedBatch`, `SignedSlashReport`, and set-keys to Serai. It is guarded by a `HandledMessageDb` dedup with an `assert!` that message ids advance by 0 or 1.

The Tributary application layer

A **Tributary** is a standalone BFT blockchain library (`code/coordinator/tributary/`) with a mempool, a provided/unsigned/signed transaction model, and block build/verify. The coordinator supplies the *application* transactions it carries.

`enum Transaction` (`code/coordinator/src/tributary/transaction.rs:133`) enumerates the app messages: `RemoveParticipantDueToDkg`, `DkgCommitments` (`:139`), `DkgShares`, `InvalidDkgShare`, `DkgConfirmed`, `CosignSubstrateBlock`, `Batch` (`:172`), `SubstrateBlock`, `SubstrateSign`, `Sign`, `SignCompleted` (`:188`), and `SlashReport` (`:195`). Each has a hand-rolled `read` / `write`, a `kind()` (Signed / Provided / Unsigned plus an ordering string), a `verify()`, and a `sign()`, with label/kind-scoped nonces.

The per-block state machine lives in `handle_application_tx()` (`code/coordinator/src/tributary/handle.rs`). It re-derives each sender's participant index, rejects data from removed or non-participant validators, and slashes on premature/future/dated attempts, duplicate data, or wrong data lengths. An `accumulate()` step gathers a supermajority (`t`, or the full `n` for DKG) of shares before emitting a `CoordinatorMessage` to the processor. Slashing is either `fatal_slash` (a DB flag that feeds slash reports) or accumulated points; note the explicit TODO that fatally-slashed nodes can still post to the chain, a residual DoS surface.

`TributarySpec` (`code/coordinator/src/tributary/spec.rs`) fixes the set: validators, their weights, a `genesis()` transcript over `(serai_block, session,`

`network)`, `t()` = ⌊2n/3⌋+1, and the `i()` / `reverse_lookup_i()` maps between validators and `Participant` ranges (accounting for removed validators).

`SignCompleted` is unsigned — a documented spam surface

`SignCompleted` is deliberately an `Unsigned` transaction so it can be de-duplicated among signers (`code/coordinator/src/tributary/transaction.rs:182-188`). Because the signature-stripping in `hash()` only applies to `Signed` transactions, its hash commits to the random nonce `R`, and the handler carries a `TODO: Confirm this signer hasn't prior published a completion`. Whether a single validator can flood distinct valid `SignCompleted` transactions for one plan (varying `R`) to bloat blocks is an open audit question, noted here honestly rather than assumed safe.

Tendermint — the consensus core

`code/coordinator/tributary/tendermint/` is a generic Tendermint state machine providing safety and liveness under **less than one-third Byzantine weight**. `TendermintMachine::message()` (`code/coordinator/tributary/tendermint/src/lib.rs`) is the message ingress; `run()` is the select loop over synced-block / queue / timeout / rebroadcast / message.

The thresholds are stake-weighted (`code/coordinator/tributary/tendermint/src/ext.rs:163-168`):

- `threshold()` = `((total_weight * 2) / 3) + 1` (`:164`) — the weight a commit must exceed;
- `fault_threshold()` = `(total_weight - threshold()) + 1` (`:167-168`).

`verify_commit` requires distinct signers whose summed weight meets the threshold (`code/coordinator/tributary/tendermint/src/ext.rs:274`). Safety mechanics:

- **Equivocation** (two conflicting messages from one validator) produces `Evidence::ConflictingMessages`, detected in the message log.
- A **persisted equivocation guard**: each sent block/round/step is written to the DB, so a reboot cannot double-sign (`block.rs`).
- **Own-message discipline**: the node executes its own messages locally before broadcasting them; if one of its own messages ever violates an invariant, the machine `panic!`s ("honest node had invalid behavior") — a deliberate fail-stop that, via the panic hook, exits the process.

Byzantine behavior by *others* is handled by evidence transactions (`TendermintTx::SlashEvidence`, an `Unsigned` tributary transaction) rather than by trusting the transport.

DKG confirmation

Once a set completes distributed key generation, the validators must jointly confirm the resulting key on Serai. `DkgConfirmer` (`code/coordinator/src/tributary/signing_protocol.rs`) performs an n-of-n **MuSig** (Schnorrkel) signature by the validators' keys to do so. The module carries extensive commentary on nonce-reuse safety — the argument being that BFT ordering prevents a preprocess/nonce from being reused across a re-executed round — plus an XOR "encryption" of cached preprocesses with a key-derived Blake2s pad. The underlying DKG and its blame protocol are documented in Distributed Key Generation; confirming that the BFT-ordering argument holds under all message interleavings is a first-order audit question.

Peer-to-peer: authentication lives at the application layer

The transport is **untrusted**. `code/coordinator/src/p2p.rs` builds a libp2p swarm from a **throwaway ed25519 keypair regenerated on every boot** (`code/coordinator/src/p2p.rs:330`, used at `:379`) — the libp2p `PeerId` is **not** bound to any validator identity. There is an explicit `TODO` about adding

authentication “to only accept connections from” validators (`code/coordinator/src/p2p.rs:376`). Gossipsub runs in `Strict` validation mode but is signed only by that ephemeral key.

Real authentication is therefore entirely at the **application layer**: Tendermint consensus messages and Tributary transactions are Ristretto-Schnorr signed and verified against the on-chain validator set. Identity = a Ristretto public key from the set; the transport `PeerId` is not it. The consequence is a genuine open question for the P2P review: whether an arbitrary internet peer can connect, gossip on set topics, and force unbounded work (large `vec![0; len]` allocations up to the reqres size cap; heartbeat responders spawning disk-heavy tasks), given the missing admission control. These are noted as concerns, consistent with the source’s own TODOs.

Cosign evaluation — detecting a chain split

Validators continuously **cosign** finalized Serai blocks, gossiping `CosignedBlock` s. The `CosignEvaluator` (`code/coordinator/src/cosign_evaluator.rs`) independently signature-checks each gossiped cosign against the relevant on-chain set keys before it can affect the “latest cosigned block” gate, and it snapshots per-network stake (refreshed on a timer, `update_stakes`).

Its safety role is to catch a **chain split**: if it observes cosigns for a *distinct* block at a height it already has, it records the conflict (`DistinctChain::set` , `code/coordinator/src/cosign_evaluator.rs:191-202`) and accumulates the stake attesting the distinct chain (`total_on_distinct_chain` , `:206`). When enough stake has cosigned a conflicting chain, the node fail-stops (halts) rather than proceed on a possibly-forked view. As with the Tendermint own-message guard, this is a deliberate availability-for-safety trade: a griefing question (can crafted cosigns wrongly halt an honest node?) sits alongside the intended one (does it reliably detect a real split?).

What the coordinator trusts

Source	Trust	Enforcement
Other validators (P2P)	Untrusted transport	App-layer Ristretto-Schnorr signatures vs the on-chain set
Processors (message-queue)	Trusted local component	Believes processor output; binds batches to the claimed network
Serai chain	Trusted for finalized state	Only finalized blocks read; cosign gossip independently signature-checked

The throughline: **the coordinator derives all authorization from application-layer signatures against on-chain state, never from the network transport** — and it converts any invariant violation into a process exit, making liveness a first-class concern of every handler that touches remote input.

External Network Integrations

`[NETWORK-CORE]` — the chain-integration libraries under `code/networks/` are the "hands" the Serai multisig uses to read each external chain and to build, sign, and broadcast transactions that move user funds. They are DEX-agnostic: they move value, they do not interpret instructions.

Each integration exists to satisfy the processor's `Network` **trait** (documented in The Processor) for one chain. This chapter covers the three shipped integrations — Bitcoin, Ethereum, Monero — and closes with the fact that the set of integratable chains is **closed**: adding one is a source fork, not a plugin.

Finality parameters at a glance

Each integration fixes how deep a block must be buried before Serai treats it as final, its dust threshold, and its estimated block time (from the processor `Network` impls):

Chain	`CONFIRMATIONS`	`DUST`	Est. block time	
Bitcoin	6	10_000 sats	600 s	`code/processor 638`
Ethereum	1 (per 32-slot epoch)	0 (TODO)	32 × 12 = 384 s	`code/processor 405`
Monero	10	1e10	120 s	`code/processor 484`

> Note the two distinct dust constants for Bitcoin: the *processor* uses `DUST = 10_000` (`bitcoin.rs:638`) as its economic floor, while the Bitcoin *wallet*

enforces the protocol dust limit `DUST = 546` when constructing transactions (`code/networks/bitcoin/src/wallet/send.rs:32`).

Bitcoin

`code/networks/bitcoin/` builds, parses, and signs Bitcoin **Taproot (P2TR key-spend)** transactions, scans blocks for received outputs, and talks to a Bitcoin Core RPC.

- **Signing** (`code/networks/bitcoin/src/crypto.rs`) is BIP-340 Schnorr wrapped as a FROST `Algorithm` . Taproot requires an even-Y public key, so the code conditionally negates: `needs_negation` (`:26`) drives a constant-time `conditional_select` on both the challenge `c` (`:72`) and the final scalar `s` (`:146`). `tweak_keys` (`code/networks/bitcoin/src/wallet/mod.rs:67-72`) applies the BIP-341 tweak and the same negation to the group key on load.
- **Sending** (`code/networks/bitcoin/src/wallet/send.rs`) signs every input committing to `Prevouts::All` (`:375`) — so the signature binds all input amounts, defeating fee/amount tampering. Change below `DUST = 546` (`:32`) is silently dropped into the fee (`:166` , `:229`).
- **Scanning / RPC** re-validates node responses: `get_block` / `get_transaction` recompute the hash and reject a mismatch, and `send_raw_transaction` recomputes the txid. Numeric fields such as `getblockcount` height are, however, trusted as returned — a lying node's effect on the scanner is a standing audit question.

Ethereum

`code/networks/ethereum/` is in two halves: Rust that builds/reads Router transactions, and the on-chain Solidity contracts they drive.

Rust side

`RouterCommand` / `SignedRouterCommand` and the FROST `RouterCommandMachine` (`code/networks/ethereum/src/machine.rs`) construct the messages the contract verifies. The critical invariant is that the Rust `msg()` must be **byte-for-byte identical** to the Solidity `abi.encode` / `abi.encodePacked` the contract hashes — any divergence makes a signature valid for a different intent, or breaks verification outright.

Deposit detection (`code/networks/ethereum/src/router.rs`, `erc20.rs`) defends against forged deposit events: `in_instructions` re-fetches the transaction and receipt and, for ERC-20, requires a matching `Transfer` to the Router of the exact amount before crediting; it keys deposits to `key_at_end_of_block`. On failure it deliberately halts (documented as sub-optimal).

The Solidity Router — access control by Schnorr signature

`code/networks/ethereum/contracts/Router.sol` has **no owner or EOA admin**. The sole authority is `seraiKey`, an x-only secp256k1 public key with fixed even parity. Every privileged action is gated by a Schnorr signature:

- `Schnorr.verify` (`code/networks/ethereum/contracts/Schnorr.sol:22-39`) implements Schnorr via the `ecrecover` trick with a fixed parity `KEY_PARITY = 27` (`:13`), reverting on `sa == 0` (`InvalidSOrA`, `:37`) or `R == 0` (`MalformedSignature`, `:39`).
- Every message includes `block.chainid` and the monotonically increasing `nonce`, giving cross-chain and replay protection: `updateSeraiKey` builds `abi.encodePacked("updateSeraiKey", block.chainid, nonce, key)` (`code/networks/ethereum/contracts/Router.sol:89`) and verifies against the *current* key before rotating; `execute` builds `abi.encode("execute", block.chainid, nonce, transactions)` (`:165`).
- `execute` **increments the nonce *before* verifying the signature** (`:169`, verify at `:171`), which the code comments say "prevents re-entrancy double spends yet allows out-of-order execution via re-entrancy." It caps `transactions.length <= 256` (`:160`). Each OutInstruction with no calls

does a bare `.call{gas: 5_000}` (`:181-183`); with calls, it deploys a **fresh `Sandbox` per transaction** and proxies through it with `gas: 350_000` (`:198-202`). `Sandbox.sol` uses a transient-storage (`tstore` / `tload`) reentrancy guard and is intentionally never cleared; a new sandbox per execute-transaction prevents cross-user approval hooking.
- `Deployer.sol` is a trustless CREATE-based deployer with a deterministic NUMS-signed deployer account (the design comments explain why identical cross-chain addresses aren't guaranteed).

Whether the nonce-before-verify design, the hardcoded call-gas figures, and the `successes` bitfield accounting are all sound is prime audit territory, flagged here without assuming the answer.

The relayer — unauthenticated by design

The Ethereum path publishes signed commands through a separate relayer (`code/networks/ethereum/relayer/src/main.rs`). It runs two TCP servers, both bound to `0.0.0.0` : a recipience server on `:20830` (`:50`) and a fetch server on `:20831` (`:79`), with an explicit `TODO: Add auth` (`:45`). The recipience server is unauthenticated: anyone who can reach `:20830` can overwrite the stored command for a nonce. The security argument is that commands are Schnorr-signed and verified **on-chain**, so a malicious relayer can *censor* or serve stale commands but cannot *forge* a valid execution — an argument that holds only under network isolation, which the report notes.

Monero

`code/networks/monero/` detects deposits by **view-key scanning** and builds/signs Monero transactions for withdrawals, satisfying the same `Network` trait as the other two chains.

The identity space is closed

The `Network` trait makes the integration *code* pluggable, but the *identity space* it plugs into is not. `ExternalNetworkId` is a fixed three-variant enum with a hand-rolled SCALE codec (`code/substrate/primitives/src/networks.rs:21-47`); the network and coin sets are fixed arrays — `EXTERNAL_NETWORKS: [_; 3]` and `NETWORKS: [_; 4]` (`:168-176`); and the coin → network mapping is hard-coded, including Ethereum yielding `{Ether, Dai}` (`ExternalNetworkId::coins` at `:131` , `ExternalCoin::network` at `:322`).

Consequently, **adding a chain is a primitives-plus-runtime fork**, not a configuration change or a runtime plugin: you must edit these primitives (and therefore the runtime) and ship a new `Network` implementation. This is the off-chain face of the boundary finding in §5 of the boundary chapter: generality off-chain stops at "external network," and even that space is enumerated, not open. The protocol specs for each integration live at `code/spec/integrations/{Bitcoin,Ethereum,Monero,Instructions}.md` .

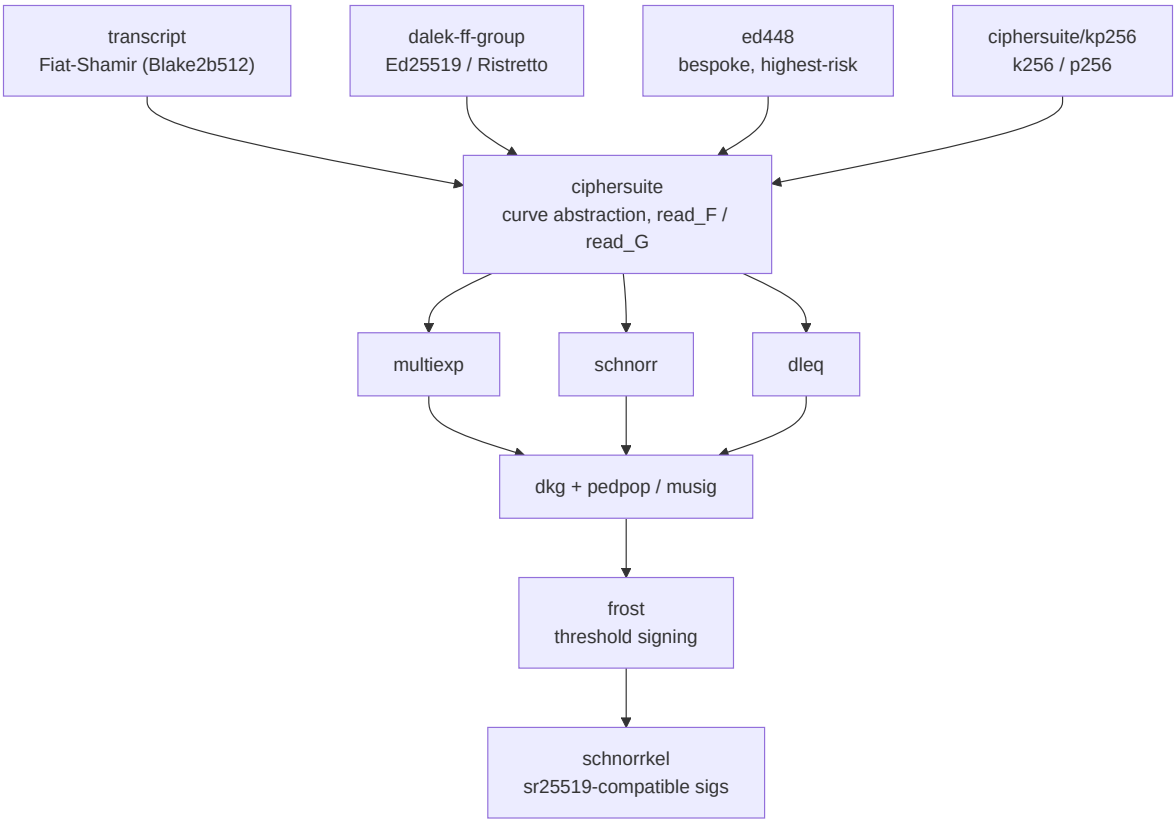
Threshold Cryptography Overview

All of `code/crypto/` *is* `[NETWORK-CORE]`. It is the custody engine of the Serai network: the from-scratch threshold-signing stack that lets a validator set act as a `t`-of-`n` multisig over Bitcoin, Ethereum, and Monero without any single party ever holding a whole key. Nothing here is DEX-specific; the same code signs every withdrawal on every chain and every Substrate extrinsic the validators co-author.

This chapter maps the stack and its shared assumptions. Distributed Key Generation and FROST Threshold Signing then go deep on the two protocols that matter most for custody.

The dependency spine

Almost every crate is generic over a `Ciphersuite` / `Curve` trait, so one family of parsing/verification routines is reused across secp256k1, P-256, Ed25519, Ristretto, and Ed448. The spine runs bottom-up:



Crate	Role
`transcript`	Fiat-Shamir transcripts. `RecommendedTranscript = DigestTranscript<Blake2b512>` (`code/crypto/transcript/src/lib.rs`). Each field is framed `tag ‖ u64-LE length ‖ value` ; `challenge()` forks the running state with distinct domain bytes as a length-extension defence.
`ciphersuite` (+ `kp256`)	Unifies a scalar field, prime-order group, and hash behind one trait; canonical (de)serialization. The **gate** for all foreign field/point bytes (`read_F` , `read_G`).
`dalek-ff-group` , `ed448`	Curve backends. `dalek-ff-group` adapts `curve25519-dalek` ; `ed448` is hand-rolled `minimal_ed448` — the most bespoke, math-heavy code in the tree.
`multiexp`	Straus/Pippenger multiexponentiation and randomized batch verification (`BatchVerifier`).

Crate	Role
`schnorr`	Strict Schnorr signatures `(R, s)` with half-aggregation.
`dleq`	Discrete-log-equality proofs (and an `experimental` cross-group DLEq).
`dkg` (+ `pedpop`, `musig`, `dealer`, `recovery`, `promote`)	Threshold-key types and the DKG protocols.
`frost`	Generic two-round FROST threshold signing; ships the IETF Schnorr instantiation for all five curves.
`schnorrkel`	A FROST `Algorithm` producing sr25519-verifiable Ristretto signatures for Substrate compatibility.

The canonical-deserialization gate

Because the whole stack is generic, essentially every untrusted scalar or point enters through two functions in `ciphersuite` (`code/crypto/ciphersuite/src/lib.rs`):

- `read_F` relies on the field's `from_repr` being canonical.
- `read_G` decodes a point, then **re-encodes it and byte-compares** to reject non-canonical encodings — the primary defence against malleable point representations.

`frost::Curve::read_G` adds one more check: it **rejects the identity point** (`code/crypto/frost/src/curve/mod.rs:125-127`). A weakness in this gate would be systemic, so it is the first thing an auditor should scrutinise across every backend (Ristretto vs Edwards canonicity, the Ed448 sign bit, k256/p256 round-trip).

The three shared assumptions

Every protocol in this stack rests on the same three foundations. They are worth stating once, because most crypto-level findings are violations of one of them:

1. **Channels are externally authenticated.** These crates do **not** authenticate senders. They assume the caller delivers each party's message over an already-authenticated channel — in Serai, that authentication is the coordinator's Tributary/Tendermint layer and its Ristretto-Schnorr signatures (see The Coordinator). Constructors like `AdditionalBlameMachine::new` explicitly assume their inputs are pre-authenticated.
2. **Callers craft binding Fiat-Shamir challenges.** `schnorr` and `dleq` push challenge construction to the caller and repeatedly warn that the challenge MUST bind key + nonce + message, or forgery is possible. `hash_to_F` naively prefixes its domain-separation tag, so the docs themselves flag a DST-transposition risk if two ciphersuites' tags can be confused.
3. **Randomness is rejection-sampled and secret-mixed.** Non-zero scalars come from `random_nonzero_F` (rejection sampling); FROST nonces additionally mix RNG entropy with the secret share (`code/crypto/frost/src/curve/mod.rs:94-112`). The single most dangerous misuse in the whole stack — reusing a FROST preprocess — is covered in FROST Threshold Signing.

Systemic risk surfaces

- **Unbounded length-prefixed reads.** Several `read` paths take a 4-byte length then allocate/loop (`SchnorrAggregate::read`, pedpop `Commitments::read`, FROST `Commitments::read`) — candidate allocation/DoS sinks when fed attacker bytes.
- **Bespoke math.** The `ed448` backend, the manual wide-reduction in `dleq::challenge`, and `kp256::hash_to_F`'s hand-rolled modulus

reduction are the highest-risk correctness targets. Note the prior Cypher Stack audit (see Audit Coverage) **excluded** `ed448` and the `dleq/experimental` feature, so those remain unreviewed by a third party.

- **Constant-timeness.** `subtle` / `zeroize` are used pervasively, but whether the "constant-time" multiexp path is genuinely data-independent under the optimizer (when fed secret scalars) is an open review question.

Where this book's prose and the repo's own `code/spec/cryptography/*` differ, the code is authoritative; the spec documents intent, the code documents behaviour.

Distributed Key Generation

[NETWORK-CORE]. Before a validator set can hold funds it must generate a threshold key **without any party ever learning the whole secret**. Serai does this with **PedPoP** — a Pedersen DKG with proofs of possession — run per validator set, per curve. This chapter covers the DKG protocol, the share-encryption and blame machinery, the key types it produces, and how the processor stores and tweaks the resulting key material.

The DKG's inter-party messages are untrusted and attacker-influenceable; their authentication comes from the coordinator layer (see Overview §The three shared assumptions and The Coordinator). This chapter documents what the crypto guarantees on its own, and where it delegates to that outer layer.

The core threshold types

`code/crypto/dkg/src/lib.rs` defines the types every protocol below produces:

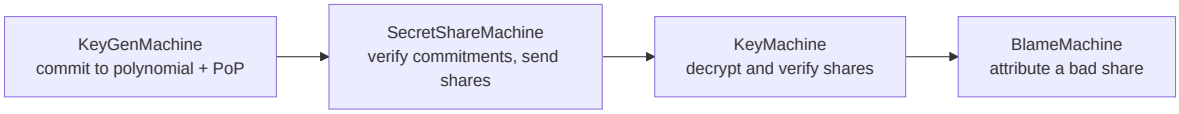
- **`Participant`** — a validated non-zero `u16` (`:26`); the borsh deserializer enforces non-zero.
- **`ThresholdParams`** — `t`, `n`, and this party's index `i`, with `0 < t <= n`, `i <= n` (`:131`).
- **`Interpolation`** (`:212`) and `interpolation_factor` (`:226`) — Lagrange interpolation, with a `Constant` mode restricted to `t == n` (used by MuSig).
- **`ThresholdKeys`** (`:292`) — the signing key material: the group key, this party's share, and the verification shares. `view()` (`:463`) sorts the included signers, rejects duplicates, enforces `t <= |included| <= n` and self-membership, and applies any offset/scale tweak.

`ThresholdKeys::read` ingests foreign key material and is trusted for signing thereafter; note that `new` recomputes the group key from the first `t` shares

but does **not** verify each verification share against a polynomial — an auditor-relevant trust edge.

PedPoP — the DKG protocol

The interactive DKG lives in `code/crypto/dkg/pedpop/src/lib.rs`, progressing through a typestate machine:



- **Round 1 — commitment + proof of possession.** Each participant commits to a degree-`t-1` polynomial and proves knowledge of its constant term with a Schnorr proof bound to the `context`, its participant index, the nonce, and the commitments (`KeyGenMachine` at `pedpop/src/lib.rs:138`; the `SchnorrSignature` PoP carried in `Commitments`, `:106,127`). `verify_r1` batch-verifies all proofs and checks commitment counts.
- **Round 2 — encrypted secret shares.** Each participant sends every other its polynomial evaluation, encrypted (below). Share validity is checked via `share_verification_statements` using the full (non-linearized) form to resist malleability.
- **Blame.** If a decrypted share is invalid, `BlameMachine` attributes fault fairly between sender and recipient using a DLEq-proven ECDH key, so a dishonest recipient cannot frame an honest sender.

Share encryption — the static-IV design

Secret shares are encrypted with a **per-message ephemeral key** (`code/crypto/dkg/pedpop/src/encryption.rs`):

- ECDH → a `RecommendedTranscript` domain-separated `"DKG Encryption v0.2"` → a ChaCha20 cipher (`cipher()` at `:101-130`).

- The IV is deliberately **static** (`:122-124`): *"reuse a nonce … Use a static IV in acknowledgement of this."* This is sound **only because each key is single-use** — the ephemeral key is fresh per message. **If that single-use property is ever violated, shares leak.** Confirming it holds across all code paths (including the test `invalidate_*` helpers and any re-encryption) is a headline auditor task.
- Each `EncryptedMessage` also carries a Schnorr **proof of possession of the ephemeral key** (`encrypt()` builds `pop_challenge` at `:157-164`) so blame cannot be abused to extract shares.

MuSig — n-of-n aggregation

`code/crypto/dkg/musig/src/lib.rs` provides non-interactive n-of-n key aggregation into `ThresholdKeys` (via `Interpolation::Constant`). Per-key **binding factors** are `hash_to_F("dkg-musig", context ‖ len ‖ keys ‖ i)` (`binding_factor` at `:81-83`), giving rogue-key resistance; `check_keys` (`:46`) rejects empty, over- `u16::MAX` , and duplicate key lists. Serai uses MuSig for **DKG confirmation**: the validators MuSig-sign, with their session keys, a message confirming the freshly-generated key on Serai, in `code/coordinator/src/tributary/signing_protocol.rs` (`DkgConfirmer`) — see The Coordinator. That file also contains the XOR "encryption" of cached preprocesses with a key-derived Blake2s pad and two unimplemented "commitments match presumed preprocess" TODOs worth review.

Key lifecycle & custody in the processor

`code/processor/src/key_gen.rs` runs PedPoP **simultaneously for two curves** per session: Ristretto (the Substrate/batch key) and `N::Curve` (the external-network key):

- **Deterministic, resumable RNG.** All key material derives from the single 32-byte `ENTROPY` env var mixed through a `RecommendedTranscript` with a per-(session, network, attempt) context (`transcript` use at `:8`), so a

rebooted processor re-derives the same DKG state rather than producing fresh (and unsafe) randomness.
- **Persistence & promotion.** Generated keys are stored in `GeneratedKeysDb` (`:36`) and promoted to `KeysDb` (`:39`) only after Substrate confirms the key pair (`ConfirmKeyPair`).
- **Custody at rest.** Key shares live in RocksDB/ParityDB at `DB_PATH` , protected only by `Zeroizing` /the `ZeroizingAlloc` global allocator — **not encrypted at rest**. This is a deliberate design choice worth stating plainly in any threat model.
- **Per-chain tweak.** `N::tweak_keys` applies network-specific key adjustments on load (`:59`), e.g. BIP-340 even-Y for Bitcoin — see FROST and External Network Integrations.

Auditor-relevant caveats

- **Static-IV single-use.** As above: the encryption's safety is entirely contingent on ephemeral keys never being reused. (`encryption.rs:122-124`)
- **Blame soundness & honest-validator removal.** Can a malicious participant craft a share that passes batch verification but yields a wrong secret, or misattribute blame to force removal of an honest validator — degrading the `t` -of- `n` fault tolerance of the resulting multisig?
- **Extra-bytes / malformed-commitment DoS.** The processor's DKG handler includes explicit malicious-extra-bytes checks; whether every malformed round message is rejected without a panic (which would exit the process — see the liveness theme in Overview) is worth confirming.
- **Unreviewed by prior audit.** The Cypher Stack crypto audit covered `/crypto` but **excluded** `ed448` ; a DKG instantiated over Ed448 inherits that gap (see Audit Coverage). `code/spec/DKG Exclusions.md` documents which participants can be excluded from a DKG and why.

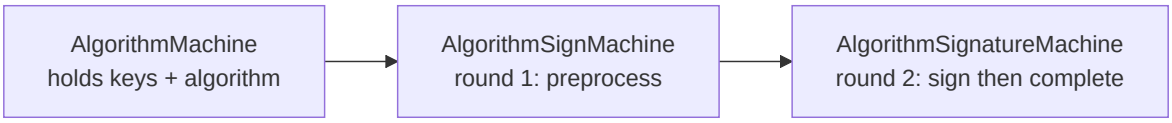
FROST Threshold Signing

[NETWORK-CORE]. Once a validator set holds a threshold key, **FROST** is how any `t` of its `n` members jointly produce a signature — a Bitcoin/Ethereum withdrawal, or a Substrate extrinsic the validators co-author — without reconstructing the secret. Serai's FROST (`code/crypto/frost/src/*`) is generic over the curve and pluggable in its `Algorithm`, shipping the IETF Schnorr instantiation for all five supported curves.

The headline safety property, stated up front because everything else serves it:

> **A FROST preprocess (nonce) MUST be used exactly once. Reuse enables third-party recovery of your private key share.**
> (`code/crypto/frost/src/sign.rs:85-92,211-214`)

The two-round machine



- **Round 1 — preprocess.** `seeded_preprocess` seeds a ChaCha20 from a 32-byte `CachedPreprocess` and generates the binomial nonces `(d, e)` per generator (`sign.rs:121-135`). `Curve::random_nonce` produces each nonce by **mixing RNG entropy with the secret share** through `hash_to_F`, then rejection-samples away zero (`code/crypto/frost/src/curve/mod.rs:94-112`) — so even a weak RNG does not trivially collapse the nonce, though it does not license reuse.
- **Round 2 — sign & complete.** Each signer broadcasts a signature share; `complete` (`sign.rs:447`) verifies the **aggregate** signature first, and only on failure runs per-share batch verification with `blame_entropy` to identify the culprit (so honest signing pays no per-share verification cost).

`sign` sorts and validates the signing set, enforces `t`, and rejects duplicate/out-of-range indices.

Nonce binding — the anti-forgery core

The binding factor `rho` is what stops Drijvers/ROS-style attacks in which a malicious signer influences another's effective nonce. It is derived from a dedicated `FROST_rho` transcript over the group key, the hashed message, and **all** signers' hashed preprocesses **before** any binding factor is computed (`sign.rs:361-379`):

```
rho_transcript = Transcript("FROST_rho")
  append "group_key" = group_key
  append "message"   = H(msg)
  append <all participants' preprocess commitments>
```

Because every signer's commitments and the message are committed before `rho` is calculated, no signer can adaptively choose its contribution to bias another's binding factor. The combined nonce is `d + e*rho`, and the `rho` transcript is then merged back into the global transcript so it stays advanced (`sign.rs:373-379`).

Why reuse is fatal — and how reboots avoid it

`CachedPreprocess` (`sign.rs:92`) exists to let a signer persist a preprocess and resume later — but the type is wrapped in `Zeroizing` and documented in the strongest terms: reuse, or third-party recovery of a cached preprocess, **recovers the private key share** (`sign.rs:83-92,209-223`). There is also an internal `unsafe_override_preprocess` (`sign.rs:147`) for testing only.

The processor honours this at the system level: when it reboots mid-signing it **deliberately aborts the attempt and waits for a fresh one** rather than reconstruct and reuse the preprocess RNG (see The Processor —

`code/processor/src/signer.rs`). Reusing the preprocess RNG across two `sign` calls is exactly the leak the type warns about.

Per-chain challenge functions (HRAm)

FROST is generic; each chain plugs in its own challenge (`Hram`) so the produced signature verifies under that chain's rules:

Chain	Hram	
Bitcoin	`code/networks/bitcoin/src/crypto.rs:56-72`	BIP-340. `needs_n` (`:25-26` even-Y n nonce is `- cx` and via const `conditi` (`:70-72`
Ethereum	`code/networks/ethereum/src/crypto.rs:118-128`	`Ethereu` challeng `keccak2` ‖ `Px` ‖ `m` a scalar; `:29-33`. match th verifier – Network
Substrate	`code/crypto/schnorrkel/src/lib.rs:41-46`	Ristretto Rebuilds Merlin `signing` **parses a context out of th** (`m[0 ..`

Chain	Hram	
		.expect message' force-set encoding `sig[63]

The IETF-transcript caveat

The default `IetfTranscript` (`code/crypto/frost/src/algorithm.rs:100-127`) is, by its own doc comment, *"incredibly naive and should not be used within larger protocols"* — it is a bare concatenation with no framing. It is safe for the IETF Schnorr instantiation shipped here, but is a trap for anyone extending the `Algorithm` surface. The `schnorrkel` algorithm deliberately uses a Merlin transcript instead.

Auditor-relevant caveats

- **Nonce/preprocess reuse (the headline).** Any path — across attempts, reboots, or coordinator message interleavings — that reuses a preprocess leaks the share. This is the sharpest custody risk in the whole stack and the subject of a dedicated focus point in the audit.
- `rho` **soundness.** Confirm every signer's commitments and the message are committed before `rho`, and that a malicious signer cannot influence another's binding factor (`sign.rs:315-380`).
- `random_nonce` **under a weak RNG.** Does the secret-share mixing still prevent reuse/leakage, and is the zero-rejection loop side-channel safe? (`curve/mod.rs:94-112`)
- **Identity rejection.** `frost::Curve::read_G` rejects the identity point (`curve/mod.rs:125-127`); the schnorrkel `m[0..4]` slice must be bounds-safe against an attacker-influenced message.

Where Serai Ends and the DEX Begins

This is the chapter the rest of the book builds toward. It answers, from the source, the question the project's own documentation never does: **which parts are the generalized Serai network, which parts are the DEX application, and where are the two wired together?**

The short answer, defended below:

> Serai *is* architected as the spec claims — a generalized primitive of **cross-chain custody plus programmable in-instructions** — and the machinery that delivers it is genuinely application-agnostic. But as **shipped**, the DEX is not merely one consumer of that machinery: it is wired back into the network's monetary policy and, critically, its **security model**. The two are cleanly separable in *concept and code layering*, and economically *inseparable at runtime*.

Every classification in this chapter is the canonical one; the per-subsystem `[NETWORK-CORE]` / `[DEX-APP]` / `[COUPLED]` tags used throughout Parts II–IV all derive from the tables here.

1. The generalized primitive, precisely

Strip everything else away and Serai's general offering is:

1. A staked validator set forms **threshold multisigs** over external chains (Bitcoin, Ethereum, Monero) — see Validator Sets and Cryptography.
2. A user's **deposit** to such a multisig is detected off-chain, and its opaque `data` payload is decoded into an **in-instruction** the Serai runtime executes **atomically on arrival**.
3. Value can be **burned** back out, producing a threshold-signed withdrawal on the external chain.

Points 1 and 3 are pure network. Point 2 — the programmable instruction — is the extension seam an "application built on Serai" would plug into. The decode is deliberately application-blind: the processor `Shorthand::decode` s the output's bytes and produces a `RefundableInInstruction` without interpreting any swap/DEX semantics (`code/processor/src/multisigs/mod.rs:66,73,91`); meaning is assigned only later, on-chain, by the runtime's `execute()` .

That is a real generalization. Its limit is equally real: **the instruction set is a closed on-chain enum**, and only one of its variants is application-free.

2. The on-chain boundary — pallet by pallet

Serai's blockchain is one `construct_runtime!` (`code/substrate/runtime/src/lib.rs:318-343`). There is **no module boundary, trait abstraction, or "application registry"** separating application pallets from network pallets — the crate name is the only signal, and the runtime ABI (`code/substrate/abi/src/lib.rs:38-59`) lists `Dex` as a first-class call variant right beside `ValidatorSets` and `InInstructions` .

Pallet	Class	Role	
`System` , `Timestamp` , `Babe` , `Grandpa` , `TransactionPayment`	NETWORK-CORE	Substrate base: system, time, consensus, finality, fees	Applicati infrastru
`Coins`	NETWORK-CORE	The ledger: native SRI + bridged balances, mint/burn/transfer, bridge-out	The shar stand on
`ValidatorSets`	NETWORK-CORE	Staking, allocation, sessions, multisig keys, slashing	Consens of the ne

Pallet	Class	Role	
`InInstructions`	NETWORK-CORE*	Executes signed cross-chain deposit batches	The instr instructi DEX (§3)
`Signals`	NETWORK-CORE	Governance: retirement (upgrades) and network halting	Applicati governar
`EconomicSecurity`	**COUPLED**	Flags when a network is economically secured	Reads th (`code/su security 46`)
`Emissions`	**COUPLED**	SRI issuance to validators & pools	Post-sec **DEX swa**
`Dex`	**DEX-APP**	The AMM: pools, swaps, liquidity — and the price oracle	The refe
`LiquidityTokens` (`coins::Instance1`)	**DEX-APP**	LP-share token accounting	Exists or
`GenesisLiquidity`	**DEX-APP**	One-time bootstrap of DEX pools + genesis SRI	Bootstra

* `InInstructions` is tagged NETWORK-CORE because the batch pipe, signature verification, sequencing and bridge-in `Transfer` are all network. But note its `Config` is compile-time bound to the DEX and its satellites: it requires `CoinsConfig + DexConfig + ValidatorSetsConfig + GenesisLiqConfig + EmissionsConfig` (`code/substrate/in-instructions/pallet/src/lib.rs:45-54`). You cannot compile the instruction pipe without the DEX in the runtime.

The externally-callable surface confirms the split. The `serai_abi::Call` enum is the exact public dispatch surface (the runtime `CallFilter` rejects anything not representable in it, `code/substrate/runtime/src/lib.rs:142-149`):

- NETWORK-CORE calls: `Coins::{transfer,burn,burn_with_instruction}`, `ValidatorSets::{set_keys,report_slashes,allocate,deallocate,claim_deallocation}`, `InInstructions::execute_batch`, `Signals::{register_retirement_signal,…}`, `Babe/Grandpa::report_equivocation`.
- DEX-APP calls: `Dex::{add_liquidity,remove_liquidity,swap_exact_tokens_for_tokens,swap_tokens_for_exact_tokens}`, `LiquidityTokens::{transfer,burn}`, `GenesisLiquidity::{remove_coin_liquidity,oraclize_values}`.
- `Emissions` and `EconomicSecurity` expose **no `Call` at all** — they are engine-room pallets driven purely by `on_initialize` hooks and internal calls.

3. The instruction interface — the boundary in one enum

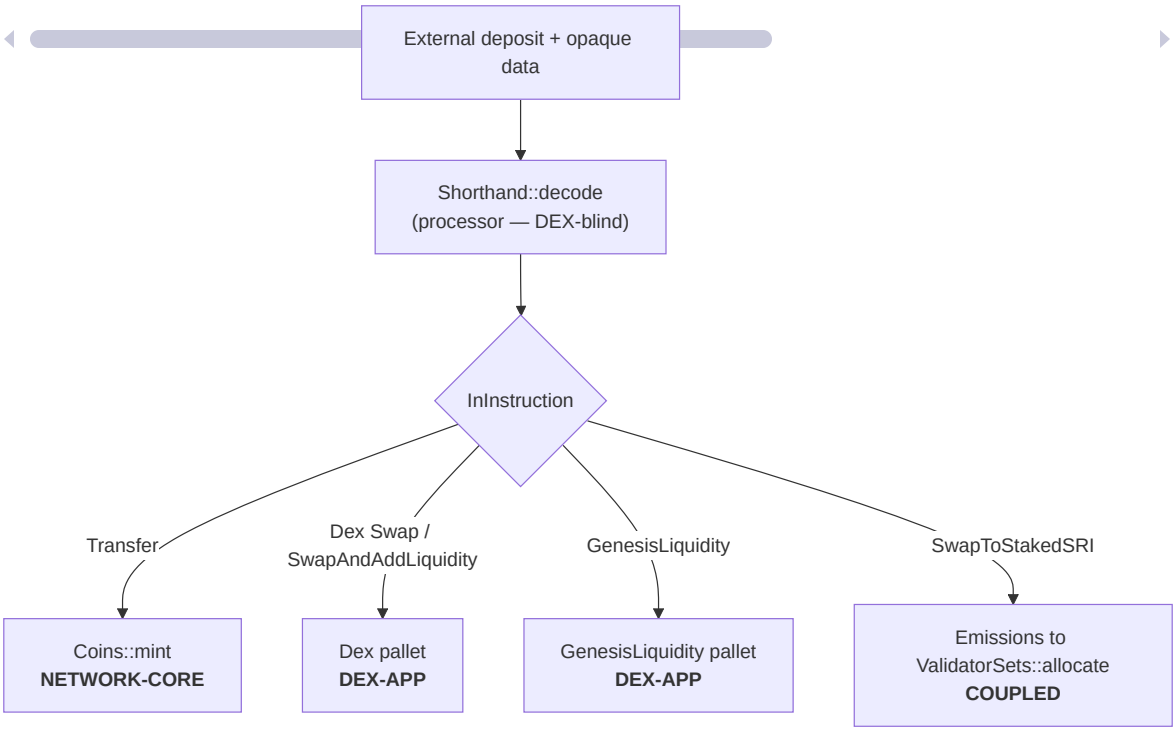
The in-instruction is where "an application built on Serai" is supposed to live. In practice, the enum hardcodes the DEX. Full classification of `InInstruction` (`code/substrate/in-instructions/primitives/src/lib.rs:77-82`) and its `execute()` dispatch (`code/substrate/in-instructions/pallet/src/lib.rs:99-230`):

Variant	Class	exe
`Transfer(address)`	**NETWORK-CORE**	`Coins::mint(ad`
`Dex(DexCall::SwapAndAddLiquidity)`	**DEX-APP**	mint → `Dex::sw` `Dex::add_liqui`
`Dex(DexCall::Swap)`	**DEX-APP**	mint → `Dex::sw` external

Variant	Class	exe
`GenesisLiquidity(address)`	**DEX-APP**	mint → `GenesisLiquidi`
`SwapToStakedSRI(address, network)`	**COUPLED**	mint → `Emissio` (DEX-priced swa

`DexCall` (`primitives/src/lib.rs:65-71`) has two variants, both DEX. The `Shorthand` sugar (`primitives/src/shorthand.rs:23-37`) has three cases — `Raw` , `Swap` , `SwapAndAddLiquidity` — of which the two DEX ones are **not even wired up yet** (`todo!()` at `shorthand.rs:51-52`); only `Raw` is functional (`:50`).

So of the whole instruction surface, exactly **one variant — `Transfer` — is application-free.** Everything else is the DEX or routes through it. The generalization is genuine but narrow: the pipe is general, the payloads shipped in it are not.



4. The coupling map — where the network depends on the application

This is the sharpest finding. The network's own security and monetary pallets **import and call the `dex` pallet.** The couplings, with exact sites:

(a) The security inversion — core bridge-mint is gated by the DEX price. The rule that decides how much foreign value a multisig may safely custody reads the DEX AMM oracle:

- The DEX produces `SecurityOracleValue` (the highest observed median price) (`code/substrate/dex/pallet/src/lib.rs:195-199,481-485`).
- `ValidatorSets::required_stake` values a coin via `Dex::security_oracle_value(coin)` (`code/substrate/validator-sets/pallet/src/lib.rs:837`), and `required_stake_for_network` sums it over supply (`:856-863`).
- `AllowMint::is_allowed` refuses to mint a bridged coin unless staked value covers that DEX-priced requirement (`:1185-1196`), and the runtime wires `coins::Config::AllowMint = ValidatorSets` (`code/substrate/runtime/src/lib.rs:204-206`). So **`Coins::mint` of any external coin can be refused because of the AMM's price** (`code/substrate/coins/pallet/src/lib.rs:167-174`).
- The same figure guards deallocation (`DeallocationWouldRemoveEconomicSecurity` , `validator-sets/pallet/src/lib.rs:610-613`).

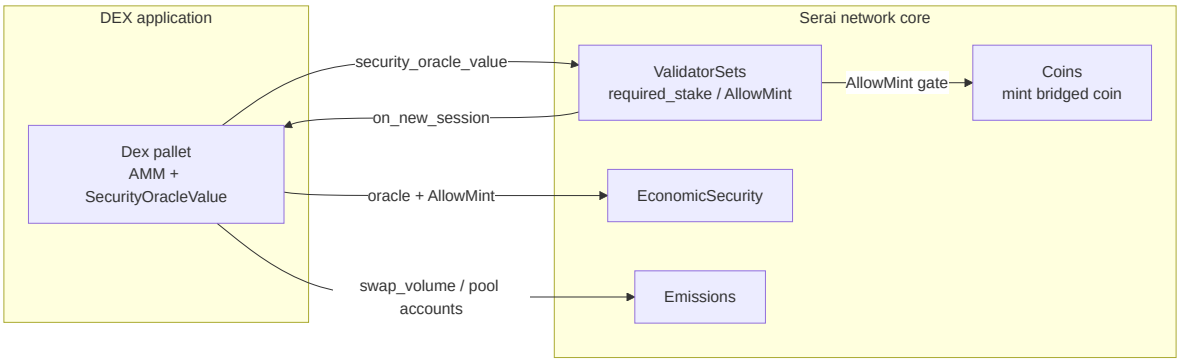
(b) EconomicSecurity is defined off the DEX. A network is "secured" only once `Dex::security_oracle_value(coin).is_some()` **and** `AllowMint::is_allowed(…)` (`code/substrate/economic-security/pallet/src/lib.rs:44-50` ; `Config: … + DexConfig` at `:20`).

(c) Emissions are split by DEX activity. Post-security SRI issuance is divided by `Dex::swap_volume` and paid into `Dex::get_pool_account` (`code/substrate/emissions/pallet/src/lib.rs:194,275`); `swap_to_staked_sri` runs a DEX swap and routes the result straight into `ValidatorSets::allocate` (`:388-422`).

(d) GenesisLiquidity bootstraps the DEX
(`Dex::add_liquidity` / `remove_liquidity` , `code/substrate/genesis-liquidity/pallet/src/lib.rs:150,300`).

(e) Shared ledger. `Coins` is minted/burned by both the core bridge path and every DEX satellite (bridge-in `:102` , bridge-out `:243-255` ; DEX arms `:114,171,215` ; genesis `:103,325` ; emissions `:274,352,419`).

(f) The session engine pokes the DEX. `ValidatorSets::new_session` calls `Dex::on_new_session` directly (`code/substrate/validator-sets/pallet/src/lib.rs:725`).



The only clean core→core mint path is `InInstruction::Transfer` , and even *that* mint passes through `AllowMint = ValidatorSets::is_allowed` , which reads the DEX oracle. **There is no configuration in which the running network is decoupled from the DEX.**

5. The off-chain stack has no application seam at all

Off-chain, the story is cleaner but the generality stops earlier. There **is** a real extension point — the `Network` trait
(`code/processor/src/networks/mod.rs:243+`) is the complete “external chain” abstraction (associated `Curve` / `Block` / `Output` / `Eventuality` / `Scheduler` , constants `CONFIRMATIONS` / `DUST` /…). To add a chain you implement it, as `bitcoin.rs` / `ethereum.rs` / `monero.rs` do. See External Network Integrations.

But there is **no "application" abstraction off-chain whatsoever.** The processor produces generic `InInstructionWithBalance` s and never mentions the DEX; the coordinator only reacts to `InInstructionsEvent::Batch` (`code/coordinator/src/substrate/mod.rs:14,143`). Off-chain generality stops at "external network"; application meaning exists only on-chain, in the closed instruction enum.

And the external-network identity space is itself **closed**: `ExternalNetworkId` is a fixed 3-variant enum with a hand-rolled codec (`code/substrate/primitives/src/networks.rs:21-47`), with fixed `EXTERNAL_NETWORKS` / `NETWORKS` arrays (`:168-176`) and hard-coded coin→network mapping (`:322-328` , `:131-137`). "Add a chain" is a primitives-plus-runtime **fork**, not a config change or plugin — the `Network` trait is pluggable in software structure, but the identity space it plugs into is not.

6. Thought experiment — building a second application on Serai today

Suppose you wanted to ship "Serai Lending" alongside the DEX, reusing the custody network. What would the code actually require?

1. **A new `InInstruction` variant** (e.g. `Lend(...)`) added to the closed enum, plus its `execute()` arm — a **runtime fork**, not an extension. The `Shorthand` decoder would need wiring too (its non- `Raw` cases are still `todo!()`).
2. **A new pallet** added to `construct_runtime!` , and `InInstructions::Config` extended to bind it — again editing core network crates.
3. **A price/valuation source for economic security.** This is the deep problem: `required_stake` only knows how to value a coin via `Dex::security_oracle_value` . Any application holding custodied value would either have to feed the *same* DEX oracle or the economic-security math would not account for it — there is no application-agnostic valuation interface.

4. **Off-chain: nothing pluggable.** The processor/coordinator don't need to know about your app (a genuine strength), but they also give you no hooks; your app is purely an on-chain runtime concern.

The missing abstractions, concretely, are: an **open instruction registry** (vs a closed enum), an **application-agnostic valuation/oracle interface** for economic security (vs a hardcoded `Dex::security_oracle_value`), and a **pallet-composition seam** that doesn't require editing `InInstructions::Config`. None of these exist today.

7. Verdict

- **Unambiguously the network:** `System` / `Timestamp` / `Babe` / `Grandpa` / `TransactionPayment`, `ValidatorSets`, `Signals`; `Coins` as the mint/burn/transfer + bridge-out ledger; the `InInstructions` pipe and its one application-free instruction `Transfer`; SRI as the native gas/stake asset. Off-chain: the generic `Network` trait, processor, coordinator, message-queue — none mention the DEX.
- **Unambiguously the DEX application:** the `Dex` pallet, `LiquidityTokens`, `GenesisLiquidity`; their ABI surfaces; `InInstruction::Dex` + both `DexCall`s + the `Swap` / `SwapAndAddLiquidity` shorthands.
- **Coupled (nominally network, DEX-dependent at runtime):** `EconomicSecurity`, `Emissions`, the `AllowMint` gate on `Coins::mint`, and `SwapToStakedSRI` — all reading the DEX oracle or DEX activity.
- **Net:** Serai is a real "cross-chain custody + programmable in-instructions" layer whose plumbing is DEX-agnostic, but whose shipped instantiation folds the DEX into its monetary policy and security model. Separable in concept; inseparable as deployed. A reader who wants "the network" gets everything in Parts II–IV **except** the AMM internals — but must understand §4's couplings to understand the network's security at all, which is why Economic Security is a first-class chapter of this book.

Networks & Coins Reference

Canonical tables for the networks Serai connects to and the coins it tracks, taken from the code at the pinned commit. The single source of truth is `code/substrate/primitives/src/networks.rs`; the project's prose table lives in `code/spec/protocol/Constants.md`. **Where the two disagree, the code is authoritative** — see Spec-vs-code discrepancies below.

Networks

`NetworkId` is `Serai` plus the three external networks (`code/substrate/primitives/src/networks.rs:80-83`, `ExternalNetworkId` at `:21-25`). Each external network operates over a specific curve; the processor generates one key set per network and binds it to the network via an additive offset (`code/spec/protocol/Constants.md:21-27`).

Network	Curve	Spec ID	Code SCALE tag	Source
Serai	Ristretto	0	`0`	`networks.rs:85-92`
Bitcoin	Secp256k1	1	`1`	`networks.rs:27-35`
Ethereum	Secp256k1	2	`2`	`networks.rs:27-35`
Monero	Ed25519	3	`3`	`networks.rs:27-35`

For networks, the spec ID and the code's `Encode` byte tag **agree**. Fixed arrays: `EXTERNAL_NETWORKS` (length 3, `:168-169`) and `NETWORKS` (length 4, `:171-176`).

Coins

`Coin` is `Serai` (native SRI) plus four external coins (`code/substrate/primitives/src/networks.rs:254-257`, `ExternalCoin` at `:193-198`). Symbols, names, and decimals come from `:330-388`.

Coin	Network	Symbol	Decimals (tracked)	Spec ID	Code SCALE tag
Serai	Serai	SRI	8	0	`0`
Bitcoin	Bitcoin	BTC	8	1	`4`
Ether	Ethereum	ETH	8	2	`5`
Dai	Ethereum	DAI	8	3	`6`
Monero	Monero	XMR	12	4	`7`

Fixed arrays: `EXTERNAL_COINS` (length 4, `:178-179`) and `COINS` (length 5, `:181-187`).

`Coin::Serai` is privileged: `Coin::native()` returns it (`:358-360`), `is_native()` matches only it (`:390-392`), and it is freely mintable as the gas/stake asset. External coins are bridge-backed and their mint is gated — see The Coins Ledger and Economic Security.

Coin-to-network mapping

A network's coins are enumerated by `ExternalNetworkId::coins()` (`code/substrate/primitives/src/networks.rs:130-137`); the reverse

(`ExternalCoin::network()`) is at `:321-328` . Ethereum is the only network with more than one coin:

Network	Coins
Bitcoin	Bitcoin
Ethereum	**Ether, Dai**
Monero	Monero

`MAX_COINS_PER_NETWORK = 8` (`:402`). The accompanying comment (`:395-401`) is an explicit design statement worth quoting, because it clarifies that Serai is **not** a token-listing platform:

> "Since Serai isn't interested in listing tokens, as on-chain DEXs will almost certainly have more liquidity, the only reason we'd have so many coins from a network is if there's no DEX on-chain … If necessary, this can be increased with a fork."

The last clause is the key network-vs-application fact: the coin/network identity space is closed and widening it is a **fork**, not a configuration change (see the boundary chapter §5).

Shared protocol types

From `code/spec/protocol/Constants.md:8-19` (cross-checked against code):

Alias	Type	Note
`SeraiAddress`	`sr25519::Public` (`[u8; 32]` wrapper)	account identifier everywhere
`Amount`	`u64`	all balances; higher-precision math promotes to `u128`
`Session`	`u32`	a validator-set epoch

Alias	Type	Note
`Validator Set`	`(NetworkId, Session)`	
`Key`	`BoundedVec<u8, 96>`	`MAX_KEY_LEN = 96` (`code/substrate/validator-sets/primitives/src/lib.rs:`
`ExternalAddress`	`BoundedVec<u8, 196>`	
`Data`	`BoundedVec<u8, 512>`	in-instruction payload cap

Spec-vs-code discrepancies

These are places where following the project's prose instead of the code would be wrong.

1. **Coin ID numbering.** `code/spec/protocol/Constants.md:40-46` numbers the coins Serai=0, Bitcoin=1, Ether=2, DAI=3, Monero=4 (a dense human-facing index). The code's SCALE `Encode` uses different byte tags: `Coin::Serai = 0`, but `ExternalCoin` encodes Bitcoin= `4`, Ether= `5`, Dai= `6`, Monero= `7` (`code/substrate/primitives/src/networks.rs:200-209,262`). So the wire byte for "Bitcoin the coin" is **4**, not the spec's 1. (The *network* tags do match the spec — the mismatch is coins only, because coin tags 1–3 are effectively reserved by the network encoding space.) Trust the code tags for any encoding/decoding work.

2. **Decimal truncation.** Ether and DAI have 18 decimals on Ethereum, but Serai tracks only **8** to keep amounts within `u64` (`networks.rs:348-354`, explicit comment); Monero is tracked at 12. On-chain amounts are therefore not the native-chain amounts — a conversion boundary the processor and bridge must respect.

3. **Economic-security multiplier.** The spec says a multisig is secure holding coins up to "33% of the Validator Set's allocated stake" (i.e. stake ≥ 3×

value, `code/spec/protocol/Validator Sets.md:1-23`), but the code's `required_stake` enforces a smaller multiple (≈ value × 1.5 + a 20% margin). The code is the binding rule and is **more permissive** than the spec's prose. This is derived in full in Economic Security; it is listed here only as a coverage flag.

Constants Reference

Consolidated protocol constants, grouped by domain, each cited to the pinned source. Blocks are the unit of time on Serai (`TARGET_BLOCK_TIME = 6` seconds); the `MINUTES` / `HOURS` / `DAYS` / `WEEKS` / `MONTHS` / `YEARS` aliases below are all measured in blocks.

Chain timing

Source: `code/substrate/primitives/src/constants.rs` .

Constant	Value	Meaning
`BLOCK_SIZE`	1 MB (`1024 * 1024`)	nominal block size
`TARGET_BLOCK_TIME`	6 seconds	target block interval
`MINUTES`	`60 / 6` = 10 blocks	derived
`HOURS`	`60 * MINUTES`	derived
`DAYS`	`24 * HOURS`	derived
`WEEKS`	`7 * DAYS`	derived
`MONTHS`	`30 * DAYS`	a month is defined as 30 days (slightly inaccurate)
`YEARS`	`12 * MONTHS`	360 days literal (~1.5% off)
`FAST_EPOCH_DURATION`	`2 * MINUTES`	tests only (fast-

Constant	Value	Meaning
		epoch feature)
`FAST_EPOCH_INITIAL_PERIOD`	`2 * FAST_EPOCH_DURATION`	tests only

Oracle / price window

Source: `code/substrate/primitives/src/constants.rs`. These size the DEX price oracle that the network's economic security depends on.

Constant	Value	Meaning	Lin
`ARBITRAGE_TIME`	`2 * HOURS`	long enough that holding a fake price up is unrealistic	:24 27
`MEDIAN_PRICE_WINDOW_LENGTH`	`2 * ARBITRAGE_TIME + 1`	median window (doubled, +1 for a true median)	:29 32

Tokenomics (SRI)

Sources: `code/substrate/primitives/src/constants.rs`, `code/substrate/emissions/primitives/src/lib.rs`. See SRI Tokenomics & Emissions for how these are applied.

Constant	Value
GENESIS_SRI	100,000,000 * 10^8 (100M SRI)
GENESIS_SRI_TRICKLE_FEED	6 * MONTHS
INITIAL_REWARD_PER_BLOCK	100,000 * 10^8 / DAYS
REWARD_PER_BLOCK	20,000,000 * 10^8 / YEARS
SERAI_VALIDATORS_DESIRED_PERCENTAGE	20
DESIRED_DISTRIBUTION	1,000
ACCURACY_MULTIPLIER	10,000
SECURE_BY	YEARS

Constant	Value
`POL_ACCOUNT`	`system_address(b"Serai-protocol_owned_liquidity")`

Validator sets & batches

Constant	Value	Meaning	
`MAX_KEY_SHARES_PER_SET`	150	cap on key shares per validator set	`code/substrate sets/primitive`
`MAX_KEY_LEN`	96	max external key length (BLS G2)	`…/validator-sets/primitive`
`MAX_BATCH_SIZE`	`25_000` (~25 kB)	max encoded size of an in-instruction `Batch`	`code/substrate instructions/p` (enforced at pal

Per-external-network finality

Each external network sets its own finality/economic constants in its `Network` impl. “Confirmed” is treated as final; a reorg past it panics (see Processor). Note Ethereum’s `CONFIRMATIONS = 1` is over 32-slot **epochs**, not single blocks.

Constant	Bitcoin
ESTIMATED_BLOCK_TIME_IN_SECONDS	600
CONFIRMATIONS	6
DUST	10,000
COST_TO_AGGREGATE	800
Source	processor/src/networks/bitcoin.rs:646

Audit Coverage

What has and has not been independently audited, **per the in-repo record only** (`code/audits/`). This is a coverage map, not a security judgement: absence of an audit is not evidence of a bug, and presence of one is not a guarantee. It matters here because a reader deciding how much to trust a given subsystem should know which parts have had external eyes.

Prior audits on record

Two audits by **Cypher Stack** are committed to the repository. Both are library-level and predate most of the runtime.

Audit	Scope	Explicit exclusions
Cypher Stack `/crypto` (March 2023)	the `/crypto` folder	the `ed448` crate, the `Ed448` ciphersuite, and the `dleq/experimental` feature
Cypher Stack `/networks/bitcoin` (August 2023)	the `/networks/bitcoin` folder (then at `/coins/bitcoin`)	—

Each audit folder also contains `Audit.pdf` (the full report) and a `LICENSE` . Provenance repos: `github.com/cypherstack/serai-audit` and `github.com/cypherstack/serai-btc-audit` .

Two caveats on applicability:

- Both audits are pinned to **2023 commits**, well before the pinned commit this book documents (`4b89cf0`). Code has changed since; the audits cover those earlier states.

- The crypto audit **excludes** exactly the most bespoke, highest-risk crypto: the hand-rolled `ed448` backend and the experimental cross-group DLEq (see Cryptography Overview).

Unaudited surface (per the in-repo record)

Everything below has **no audit on record** in `code/audits/`. This includes the entire on-chain runtime and the whole off-chain stack.

Area	Chapter	On-record audit?
Substrate runtime & ABI	Runtime & Call Surface	No
`coins` pallet (the ledger)	The Coins Ledger	No
`dex` pallet (AMM + oracle)	(DEX-APP — boundary)	No
`validator-sets` pallet	Validator Sets	No
`economic-security` + `AllowMint` gate	Economic Security	No
`emissions`, `genesis-liquidity`	SRI Tokenomics & Emissions	No
`in-instructions` pallet	Cross-Chain Instruction Interface	No
`signals` pallet (governance/halt)	Governance & Protocol Changes	No
Coordinator (Tributary, Tendermint, P2P)	The Coordinator	No
Processor (scanning, signing, rotation)	The Processor	No
`message-queue`	Overview & the Message Bus	No

Area	Chapter	On-record audit?
Ethereum & Monero integrations	External Network Integrations	No
`ed448` , cross-group DLEq	Cryptography Overview	No (explicitly excluded above)

The audited fraction is narrow: the composable crypto primitives (minus ed448/experimental) and the Bitcoin integration, both at 2023 states. **The value-bearing on-chain logic — bridging, minting, staking, economic security, the AMM, and consensus — has no external audit on record.**

Bug bounty

Serai runs an Immunefi bug-bounty program; in-scope critical vulnerabilities are rewarded up to **$30,000**, with out-of-scope reports handled at the program managers' discretion (`code/README.md:48-55`).

Source Map

A navigation index from the target's source tree to the chapters that document it, so a reader holding a `code/…` path can jump straight to its explanation. Paths are relative to the submodule at the pinned commit `4b89cf0`. The repo's own top-level layout is described at `code/README.md:10-46`.

On-chain — `code/substrate/`

Source path	Class	Documented in
`substrate/runtime`, `substrate/abi`	NETWORK-CORE	Runtime & Call Surface
`substrate/coins` (+ `LiquidityTokens` instance)	NETWORK-CORE (LiquidityTokens is DEX-APP)	The Coins Ledger
`substrate/validator-sets`	NETWORK-CORE	Validator Sets
`substrate/economic-security`	COUPLED	Economic Security
`substrate/emissions`	COUPLED	SRI Tokenomics & Emissions
`substrate/genesis-liquidity`	DEX-APP	SRI Tokenomics & Emissions (bootstrap)
`substrate/in-instructions`	NETWORK-CORE	Cross-Chain Instruction Interface
`substrate/signals`	NETWORK-CORE	Governance & Protocol Changes
`substrate/dex`	**DEX-APP**	The Boundary — documented at its

Source path	Class	Documented in
		interface only; AMM internals out of scope
`substrate/primitives`	shared	Networks & Coins, Constants
`substrate/node`, `substrate/client`	off-chain tooling	node/consensus wiring & the Rust SDK; noted in Overview, not deeply documented

Off-chain — services

Source path	Class	Documented in
`processor/`	NETWORK-CORE	The Processor
`processor/src/networks/mod.rs` (the `Network` trait)	NETWORK-CORE	Processor, External Networks
`coordinator/` (incl. `tributary`, `tributary/tendermint`)	NETWORK-CORE	The Coordinator
`message-queue/`	NETWORK-CORE	Overview & the Message Bus

External integrations — code/networks/

Source path	Class	Documented in
`networks/bitcoin`	NETWORK-CORE	External Network Integrations

Source path	Class	Documented in
`networks/ethereum` (incl. `contracts/*.sol`, `relayer`)	NETWORK-CORE	External Network Integrations
Monero integration	NETWORK-CORE	External Network Integrations
per-chain FROST HRAm (`crypto.rs` in each)	NETWORK-CORE	FROST Threshold Signing

Cryptography — `code/crypto/`

Source path	Documented in
`crypto/transcript`, `crypto/ciphersuite` (+ `kp256`)	Cryptography Overview
`crypto/dalek-ff-group`, `crypto/ed448`	Cryptography Overview
`crypto/schnorr`, `crypto/dleq`, `crypto/multiexp`	Cryptography Overview
`crypto/dkg` (+ `pedpop`, `musig`)	Distributed Key Generation
`crypto/frost`, `crypto/schnorrkel`	FROST Threshold Signing

Shared utilities — `code/common/`

`common/*` are utility crates used across the whole tree; they are referenced from the chapters that use them rather than given their own chapter:

Crate	Role	Referenced from
`common/db`	`Get` / `DbTxn` / `Db` traits, `create_db!` / `db_channel!`	Processor, Coordinator, Message Bus

Crate	Role	Referenced from
`common/request`	HTTP client used by RPC transports	External Networks
`common/zalloc`	zeroizing global allocator	Cryptography Overview, Processor
`common/env`, `common/std-shims`, `common/patchable-async-sleep`	env/secret loading, `no_std` shims, patchable sleep	noted where relevant

Intentionally not documented

These are out of scope for a *network* manual and are not covered here:

- **DEX AMM internals** (`substrate/dex` pool math, LP accounting, swap routing) — this book documents the DEX only as the reference application at its interface. See The Boundary.
- `tests/`, `orchestration/`, `mini/`, `patches/` — test harnesses, Docker/deploy scaffolding, and build patches, not protocol logic.